

Seasonal Autoregressive Integrated Moving Average Process (SARIMA)

Course: Practical Time Series Analysis

Lecturer: Dr. David Zmiaikou

June 4, 2024

Contents

- 01 Autoregressive Integrated Moving Average Process (ARIMA)**

- 02 Seasonal Autoregressive Integrated Moving Average Process (SARIMA)**

- 03 Identifying Seasonal Patterns in a Time Series**

- 04 Identifying a Seasonal Autoregressive Integrated Moving Average Process**

- 05 Case Study: Forecasting the number of monthly air passengers**

 - a with ARIMA**

 - b with SARIMA**

01

Autoregressive Integrated Moving Average Process (ARIMA)

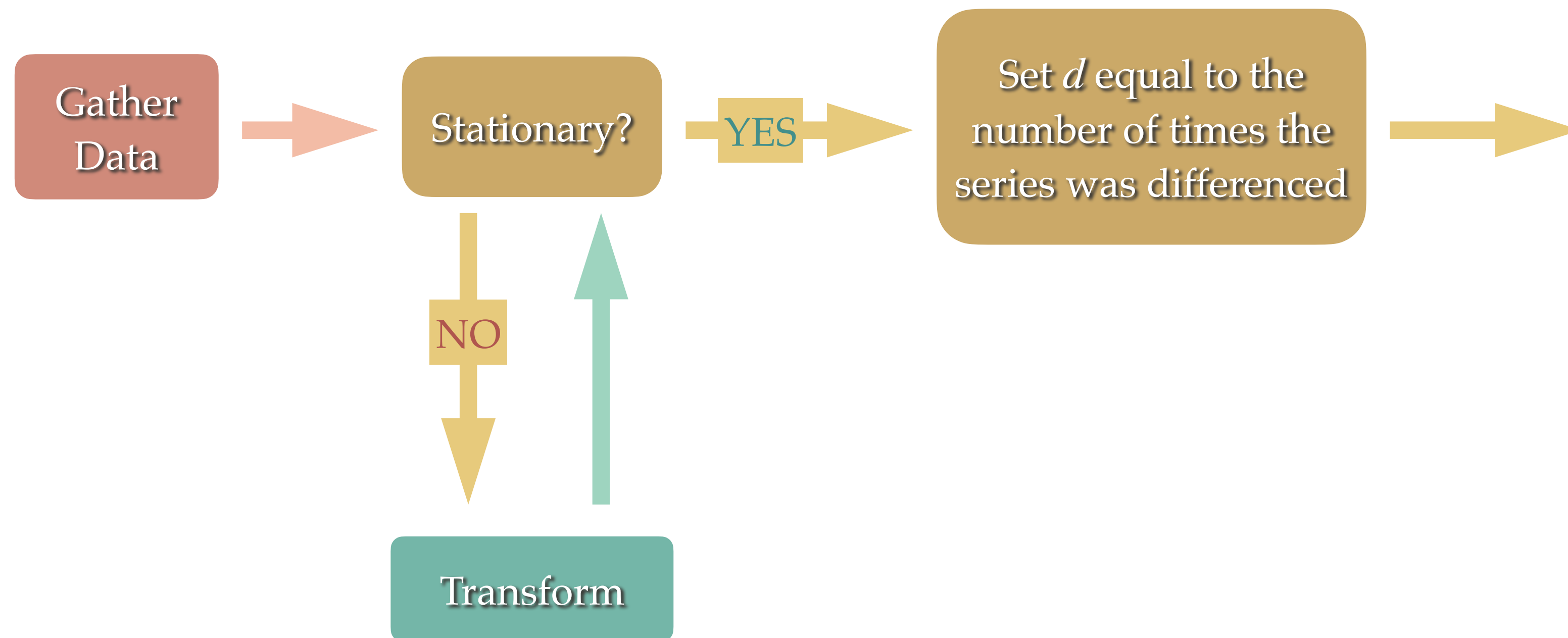
def What is an Autoregressive Moving Integrated Average Process (ARIMA)?

- ❖ An **autoregressive integrated moving average (ARIMA)** process is the combination of the $AR(p)$ and $MA(q)$ processes, but in terms of the differenced series.
- ❖ It is denoted by $ARIMA(p, d, q)$, where p is the **order** of the autoregressive part, d is the **order of integration**, and q is the **order** of the moving average part.
- ❖ Integration is the reverse of differencing, and the **order of integration** d is equal to the number of times the series has been differenced to be rendered stationary.
- ❖ The general expression is

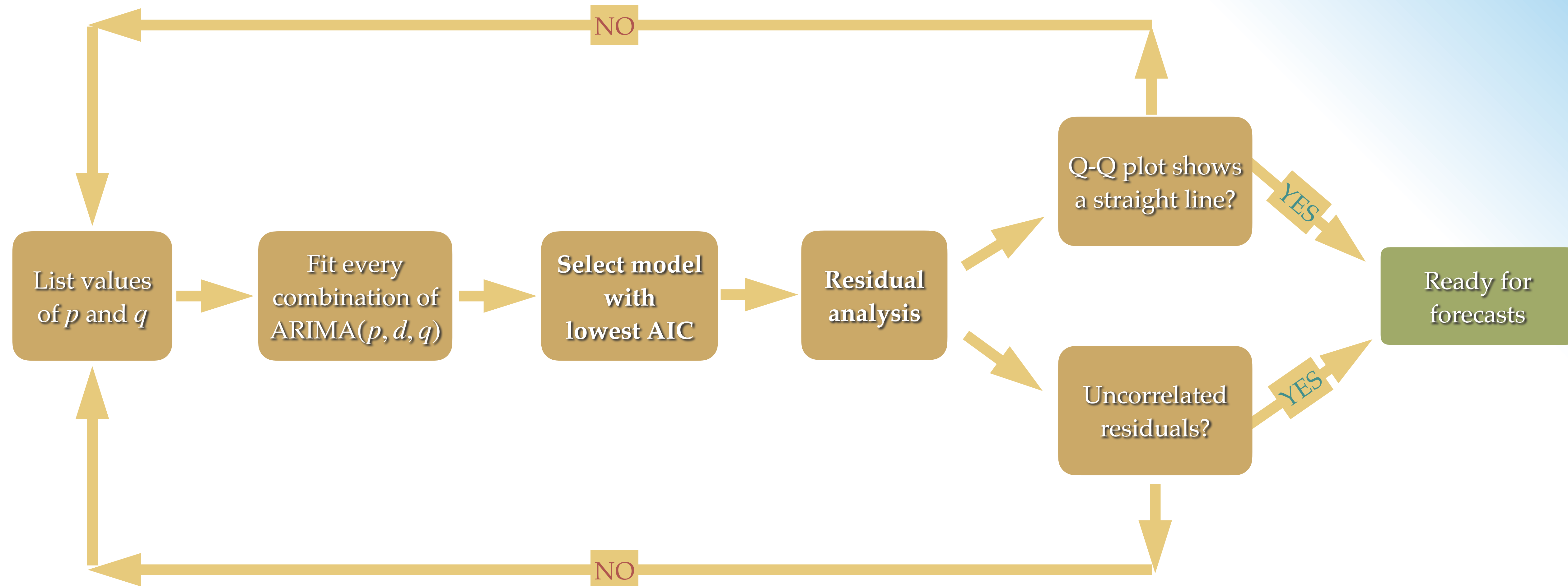
$$y'_t = C + \phi_1 y'_{t-1} + \phi_2 y'_{t-2} + \cdots + \phi_p y'_{t-p} + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q}$$

where y'_t represents the differenced series, and it may have been differenced more than once.

Identifying an Autoregressive Integrated Moving Average Process



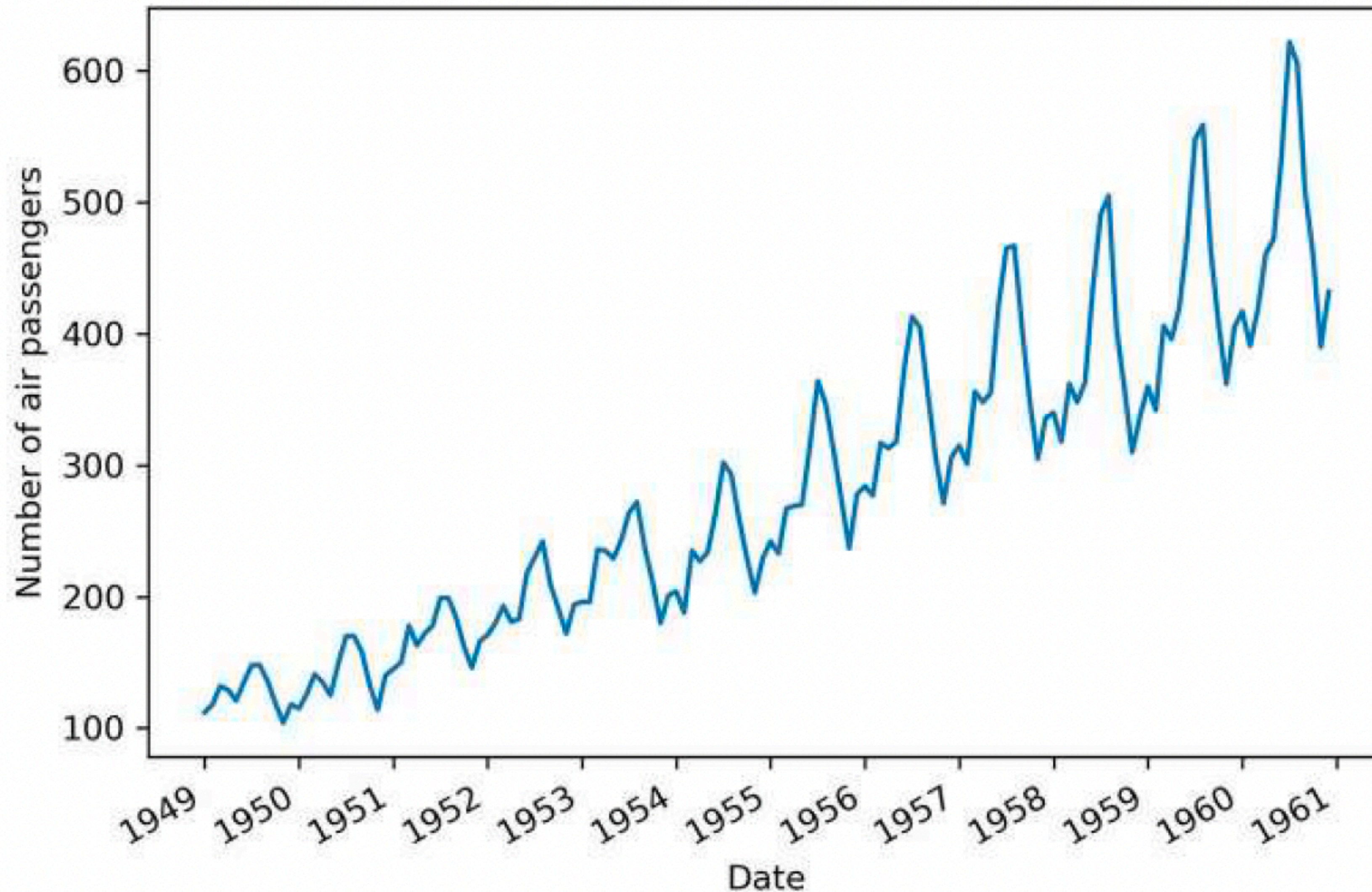
Identifying an Autoregressive Integrated Moving Average Process



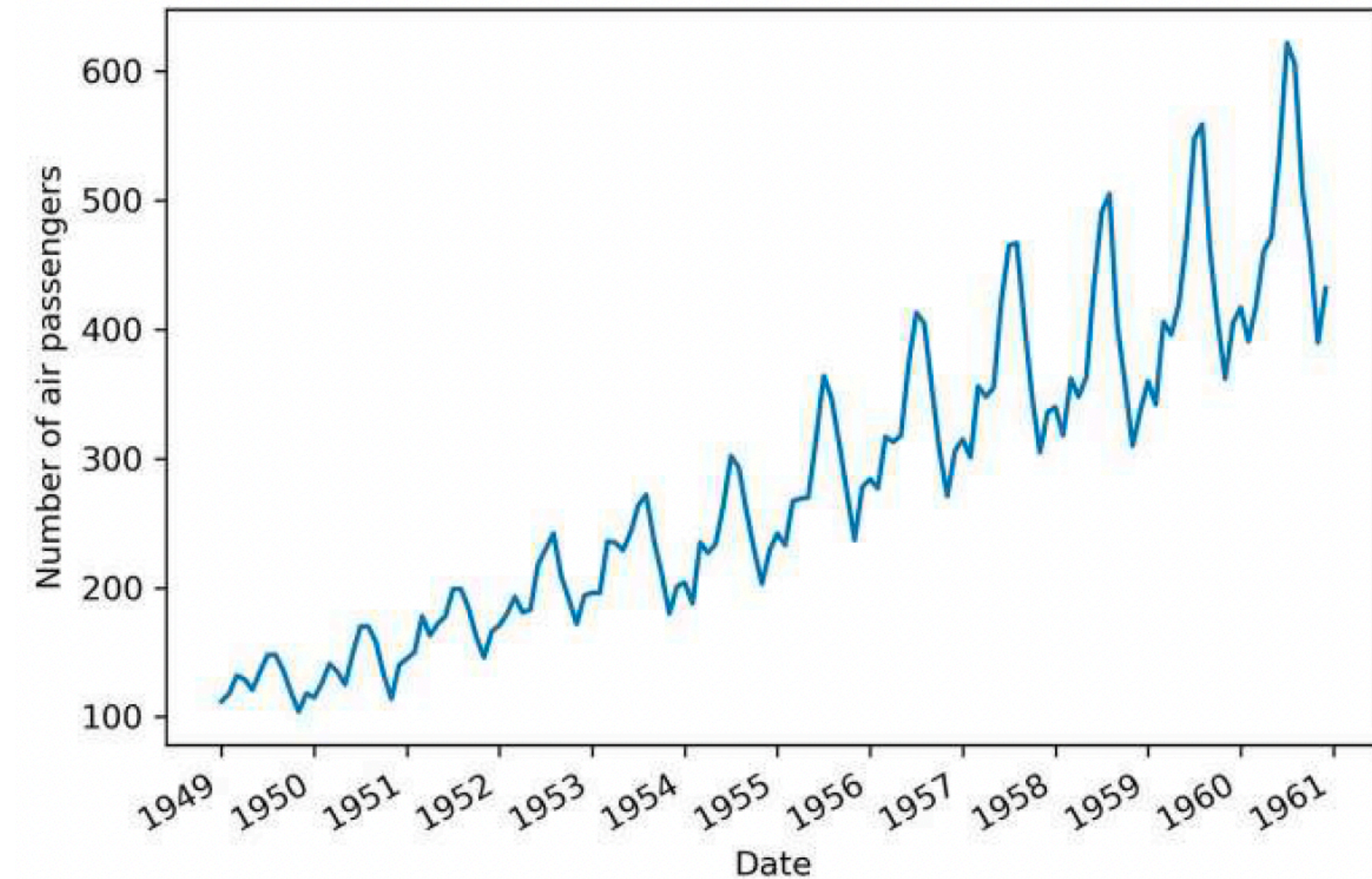
02

Seasonal Autoregressive Integrated Moving Average Process (SARIMA)

Example 1: Monthly total number of air passengers for an airline, from January 1949 to December 1960



Example 1: Monthly total number of air passengers for an airline, from January 1949 to December 1960



- Long-term trend.
- Not stationary.
- A clear cyclical pattern.
- The number of air passengers is lower at the beginning and end of the year, and it spikes up during the months of June, July, and August.

^{def} What is an Seasonal Autoregressive Integrated Moving Average Process (SARIMA)?

- ❖ The **seasonal autoregressive integrated moving average** (SARIMA) model adds seasonal parameters to the $ARIMA(p, d, q)$ model.
- ❖ It is denoted as $SARIMA(p, d, q)(P, D, Q)_m$, where
 - P is the order of the seasonal $AR(P)$ process,
 - D is the seasonal order of integration,
 - Q is the order of the seasonal $MA(Q)$ process,
 - m is the **frequency** — the number of observations per seasonal cycle.
- ❖ Note that a $SARIMA(p, d, q)(0,0,0)_m$ model is equivalent to an $ARIMA(p, d, q)$ model.

Frequency

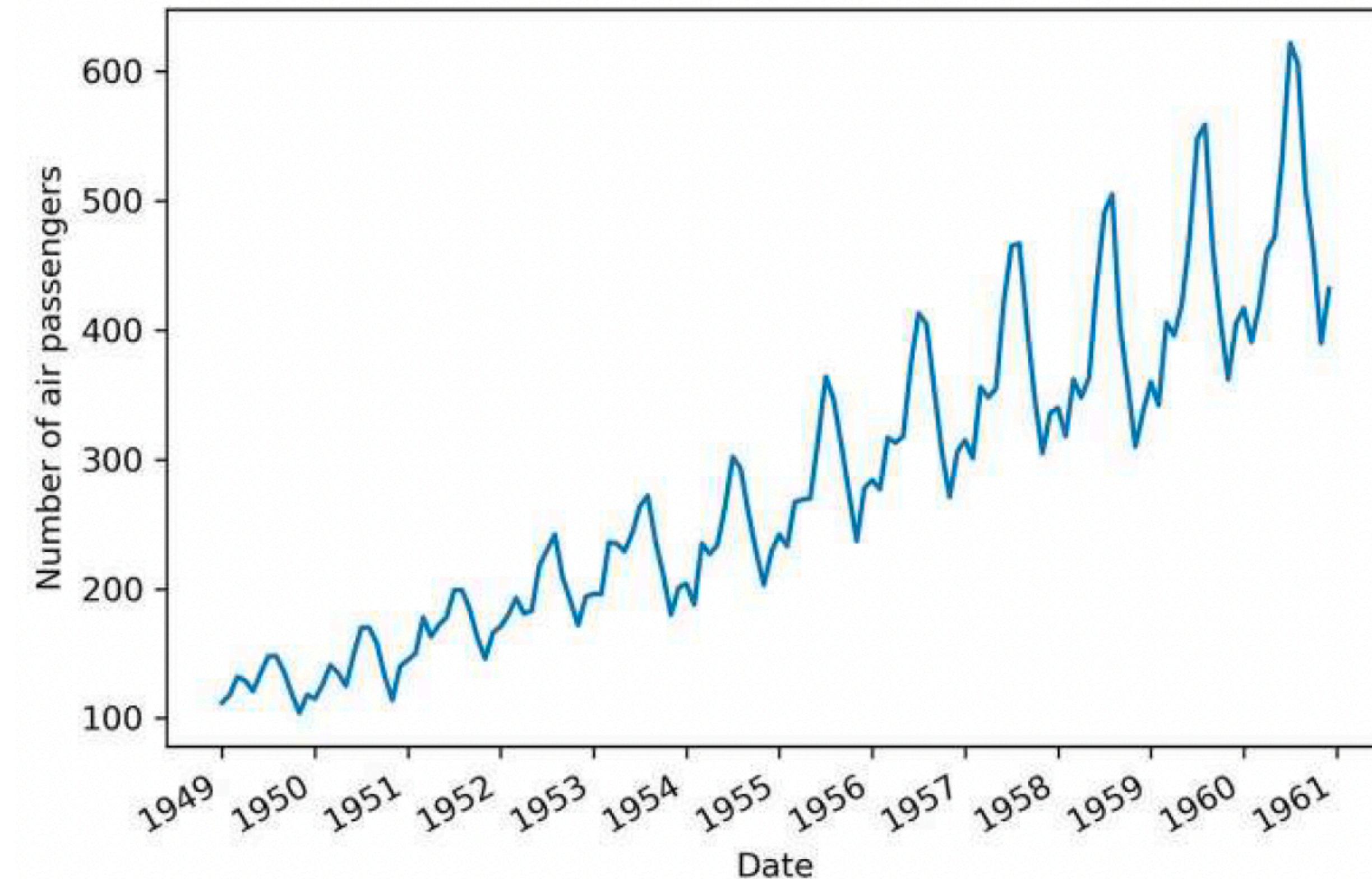
Data collection	Frequency <i>m</i>
Annual	1
Quarterly	4
Monthly	12
Weekly	52

Data collection	Frequency <i>m</i>				
	Minute	Hour	Day	Week	Year
Daily				7	365
Hourly			24	168	8766
Every minute		60	1440	10080	525960
Every second	60	3600	86400	604800	31557600

^{def} What is an Seasonal Autoregressive Integrated Moving Average Process (SARIMA)?

- ❖ Let's consider an example for $\text{SARIMA}(p, d, q)(P, D, Q)_m$ when $m = 12$. If $P = 2$, this means that we are including two past values of the series at a lag that is a multiple of m . Therefore, we'll include the values at y_{t-12} and y_{t-24} .
- ❖ Similarly, if $D = 1$, this means that a seasonal difference
$$y'_t = y_t - y_{t-12}$$
makes the series stationary.
- ❖ In a situation where $Q = 2$, we'll include past error terms at lags that are a multiple of m : the errors ϵ_{t-12} and ϵ_{t-24} .

Example 1: Monthly total number of air passengers for an airline, from January 1949 to December 1960

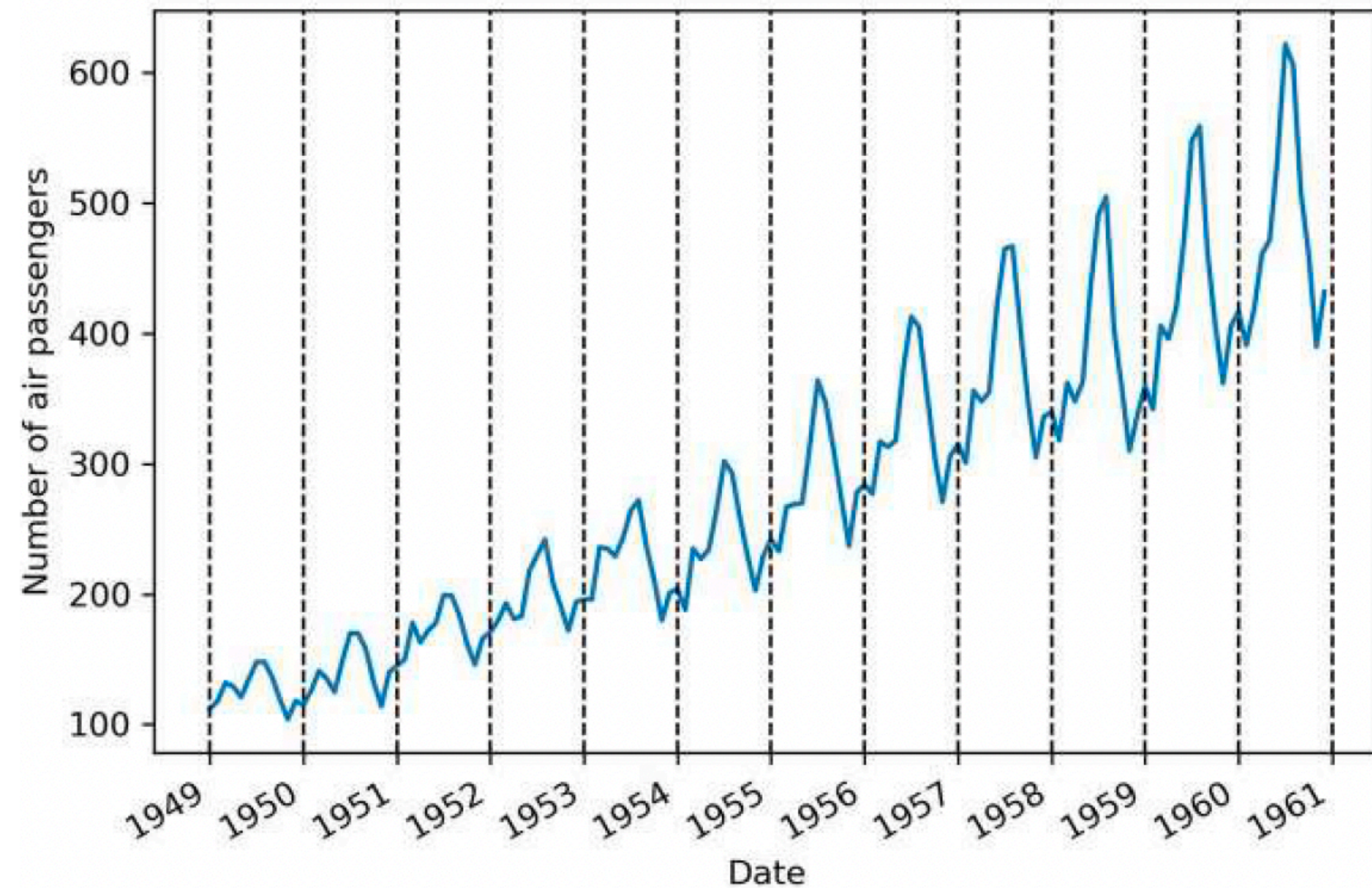


- We know that this is monthly data, which means that $m = 12$.
- Also, we can see that the months of July and August usually have the highest numbers of air passengers in the year.
- Therefore, if we are to forecast the month of July in 1961, the information coming from the month of July in prior years is likely going to be useful, since we can intuitively expect the number of air passengers to be at its highest point in the month of July 1961.
- The parameters P , D , Q and m allow us to capture that information from the previous seasonal cycle to help us forecast our time series.

03

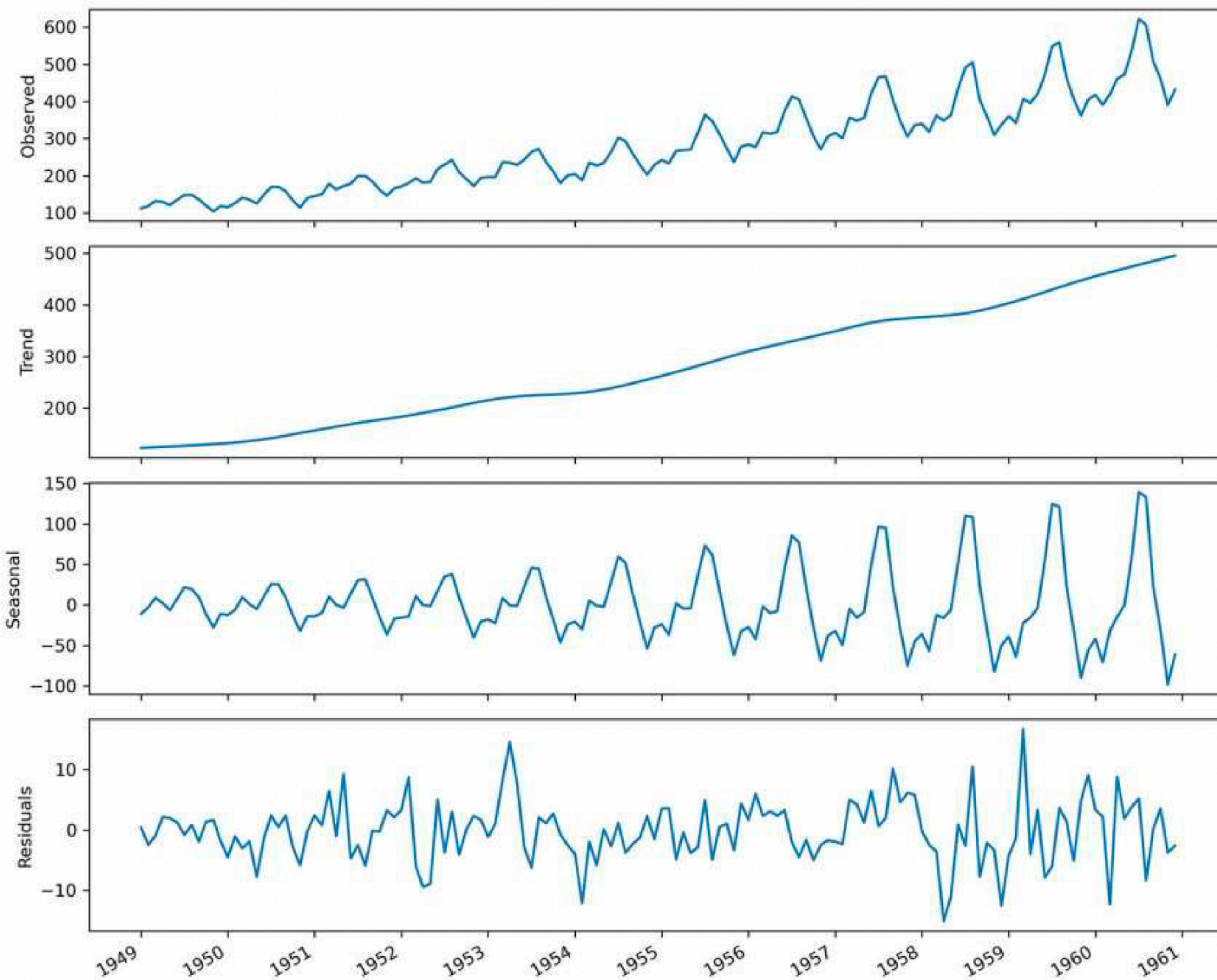
Identifying Seasonal Patterns in a Time Series

Method 1: Plot the time series



- We can clearly see how a peak occurs in the middle of each year, and there is a very similar pattern for the beginning and end of each year.
- This observation is usually enough to determine that the dataset is seasonal.

Method 2: Time series decomposition



- **Time series decomposition** is a statistical task that separates the time series into its three main components: a trend component, a seasonal component, and the residuals.
- The **trend component** represents the long-term change in the time series. This component is responsible for time series that increase or decrease over time.
- The **seasonal component** is the periodic pattern in the time series. It represents repeated fluctuations that occur over a fixed period of time.
- The **residuals**, or the noise, express any irregularity that cannot be explained by the trend or the seasonal component.

Method 2: Time series decomposition

- Use the **STL** function from **statsmodels**:

```
from statsmodels.tsa.seasonal import STL
```

```
decomposition = STL(df['Passengers'], period=12).fit()
```

```
fig, (ax1, ax2, ax3, ax4) = plt.subplots(nrows=4, ncols=1,  
sharex=True, figsize=(10,8))
```

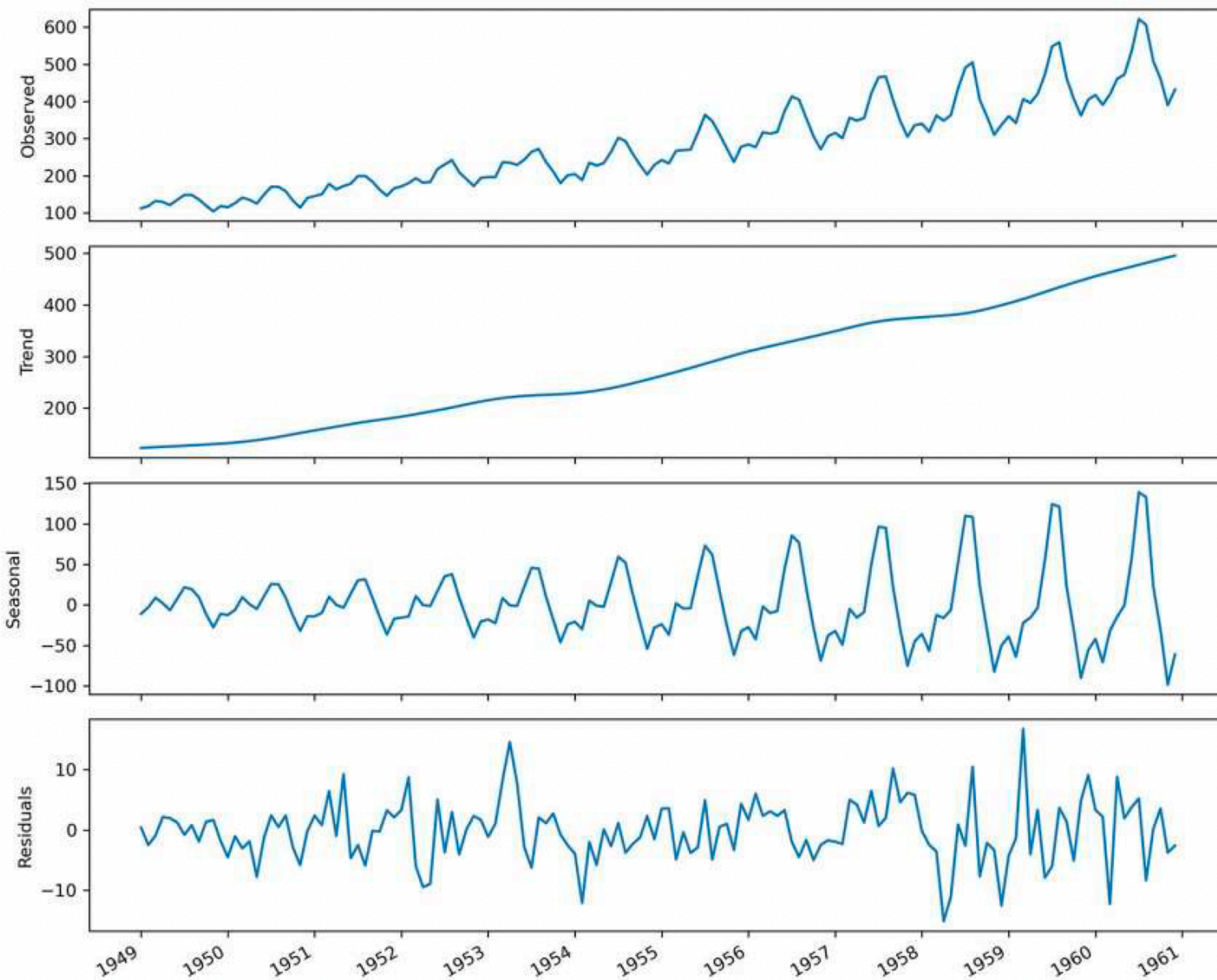
```
ax1.plot(decomposition.observed)  
ax1.set_ylabel('Observed')
```

```
ax2.plot(decomposition.trend)  
ax2.set_ylabel('Trend')
```

```
ax3.plot(decomposition.seasonal)  
ax3.set_ylabel('Seasonal')
```

```
ax4.plot(decomposition.resid)  
ax4.set_ylabel('Residuals')
```

```
plt.xticks(np.arange(0, 145, 12), np.arange(1949, 1962, 1))
```



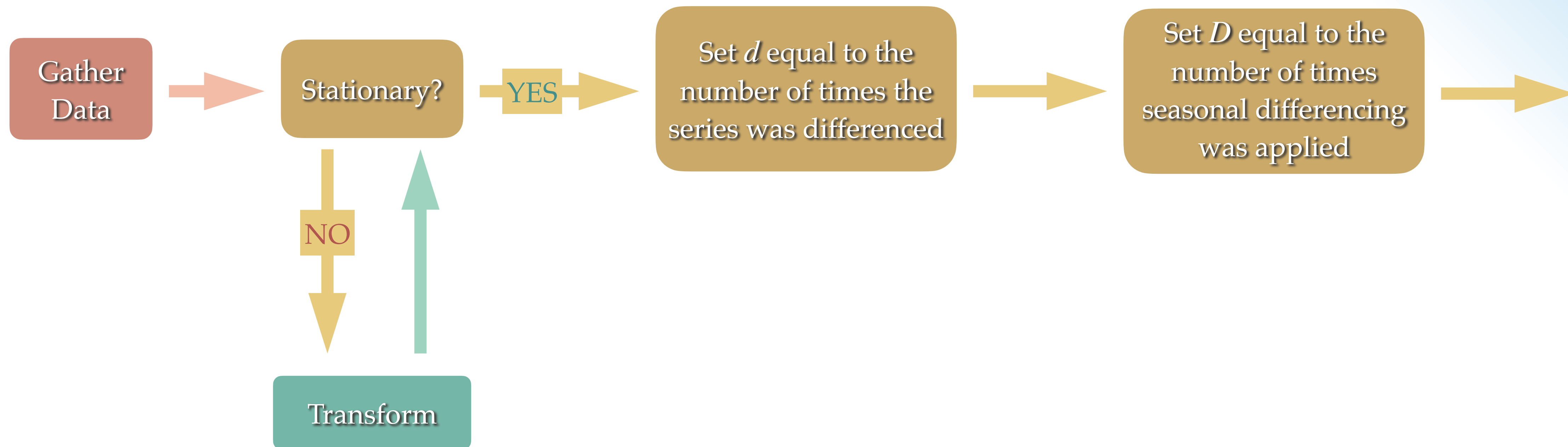
Method 3: A statistical test?

- To my knowledge, there are no statistical tests to identify seasonality in time series.
- Yet to be developed.

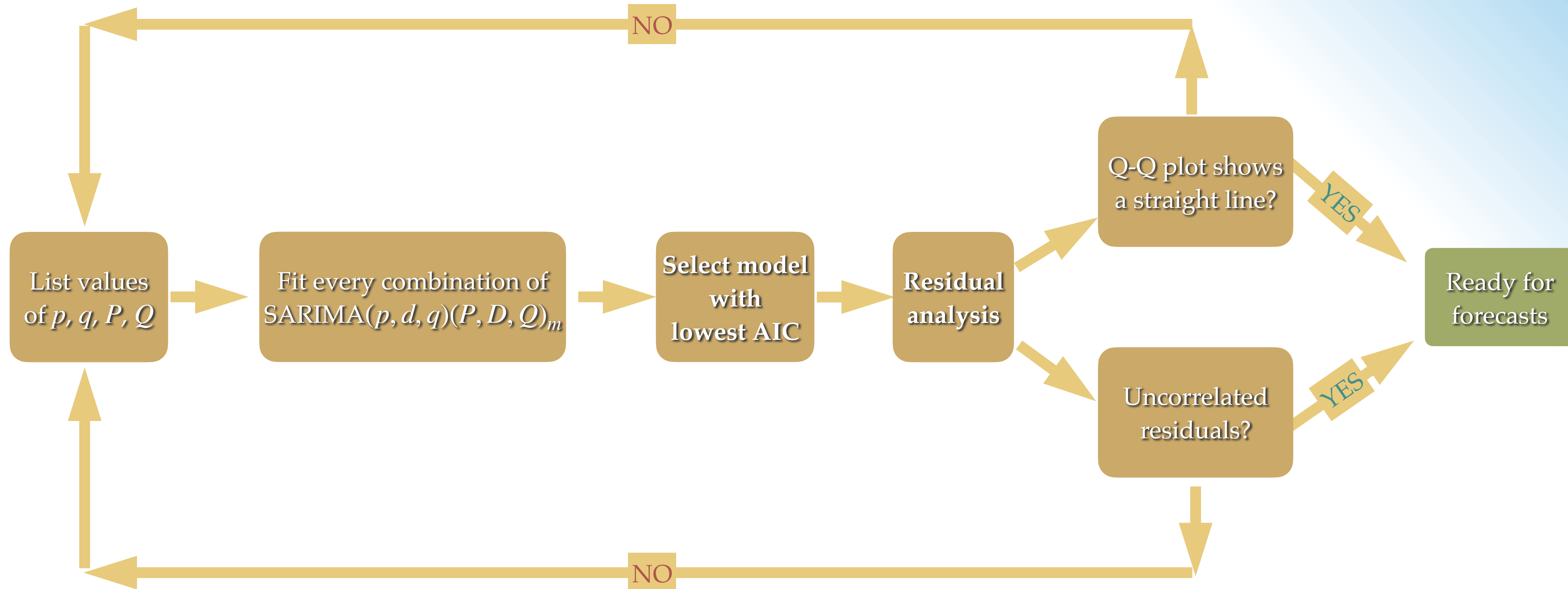
04

Identifying a Seasonal Autoregressive Integrated Moving Average Process

Identifying a Seasonal Autoregressive Integrated Moving Average Process



Identifying a Seasonal Autoregressive Integrated Moving Average Process



05

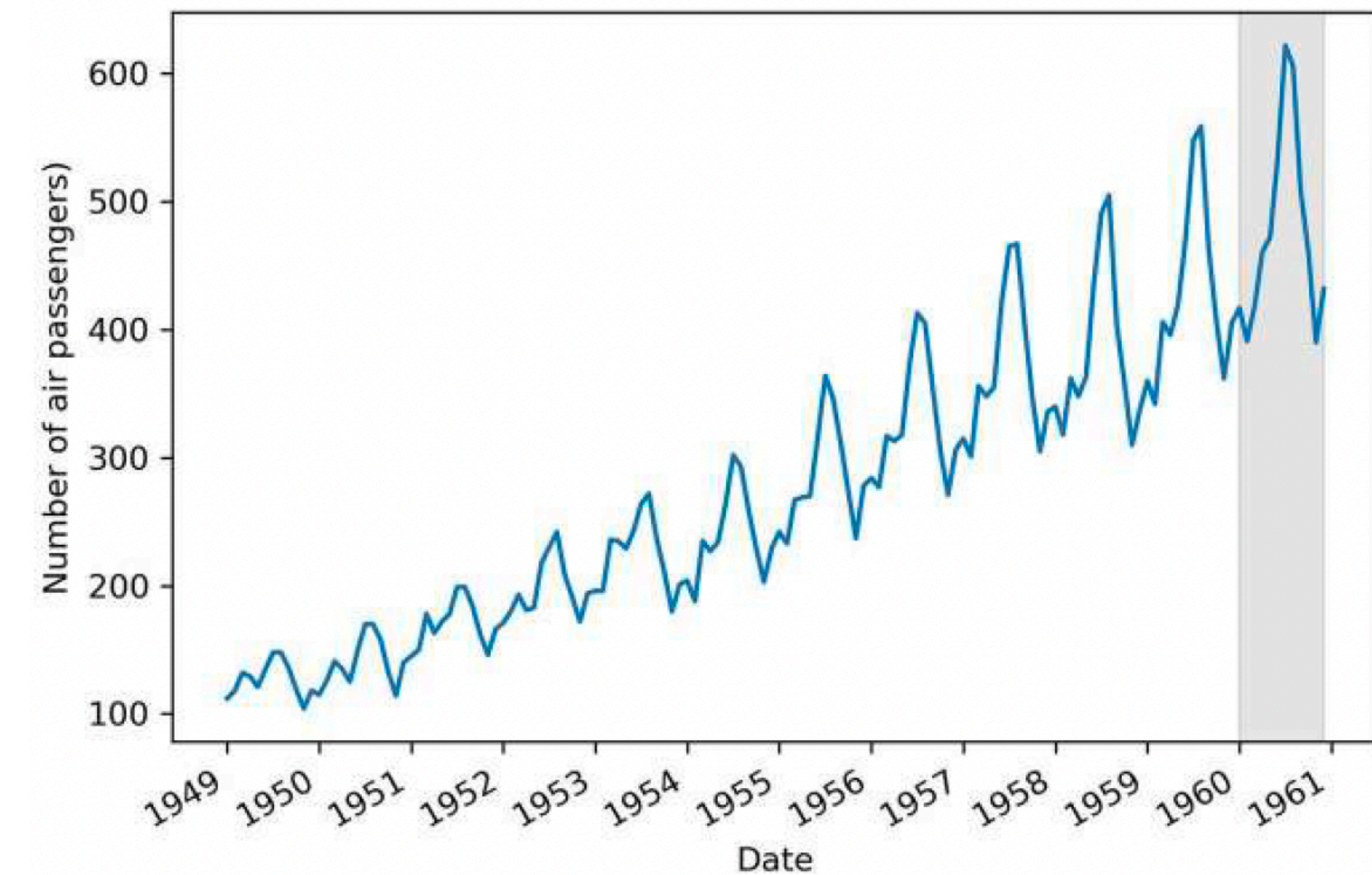
Case Study: Forecasting the number of monthly air passengers

Forecasting the number of monthly air passengers

- ❖ The baseline model will be the **naive seasonal forecast**, and we will use both the $\text{ARIMA}(p, d, q)$ and $\text{SARIMA}(p, d, q)(P, D, Q)_m$ models to verify whether the addition of seasonal components will yield better forecasts.

ARIMA

Number of monthly air passengers (ARIMA): Stationarity

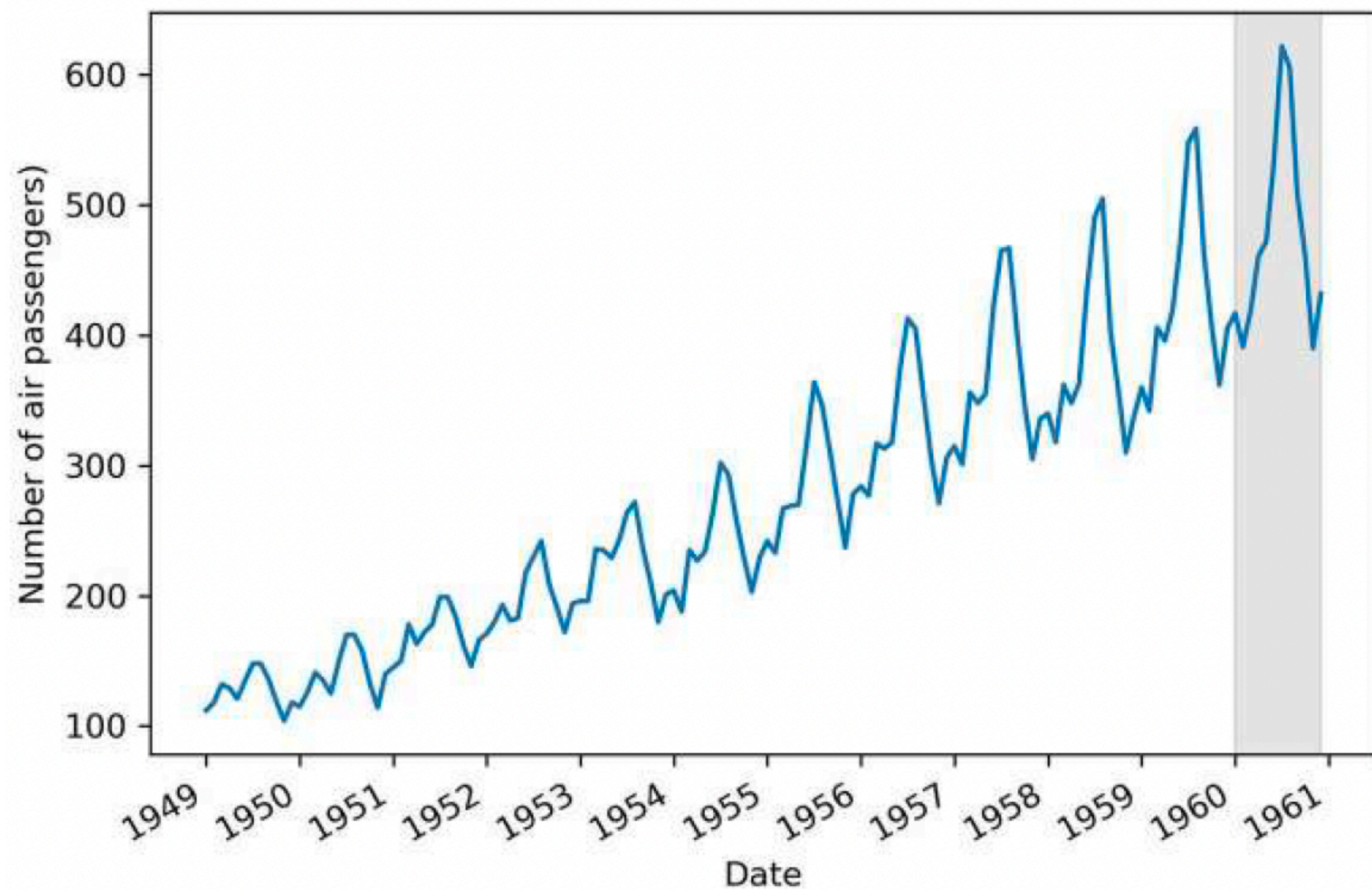


- Use the **adfuller** function from **statsmodels**:

```
from statsmodels.tsa.stattools import adfuller  
ADF_result = adfuller(df['Passengers'])  
print(f'ADF Statistic: {ADF_result[0]}')  
print(f'p-value: {ADF_result[1]}')
```

This results in an ADF statistic of 0.82 and a p-value of 0.99.
Not stationary.

Number of monthly air passengers (ARIMA): Stationarity

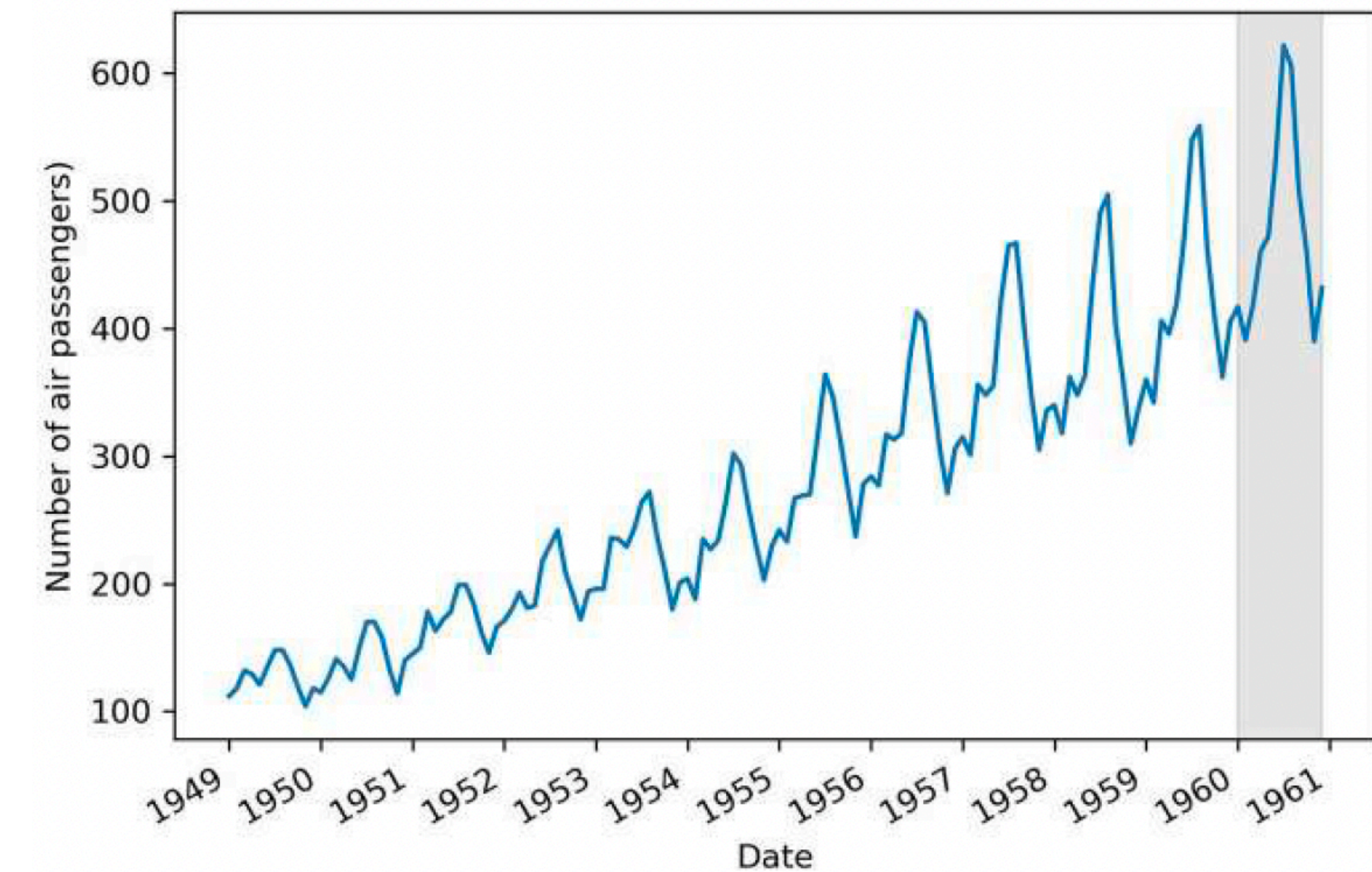


- Apply differencing:

```
df_diff = np.diff(df['Passengers'], n=1)  
ADF_result = adfuller(df_diff)  
print(f'ADF Statistic: {ADF_result[0]}')  
print(f'p-value: {ADF_result[1]}')
```

This results in an ADF statistic of -2.83 and a p-value of 0.054 .
Not stationary.

Number of monthly air passengers (ARIMA): Stationarity



❖ Again apply differencing:

```
df_diff2 = np.diff(df_diff, n=1)  
ADF_result = adfuller(df_diff2)  
print(f'ADF Statistic: {ADF_result[0]}')  
print(f'p-value: {ADF_result[1]}')
```

This results in an ADF statistic of -16.38 and a p-value of 2.73×10^{-29} . *Stationary.*

❖ Hence, $d = 2$

def What is Akaike information criterion (AIC)?

Number of model
parameters

$$AIC = 2\ln\left(\frac{e^k}{\hat{\mathcal{L}}}\right)$$

Maximized Likelihood

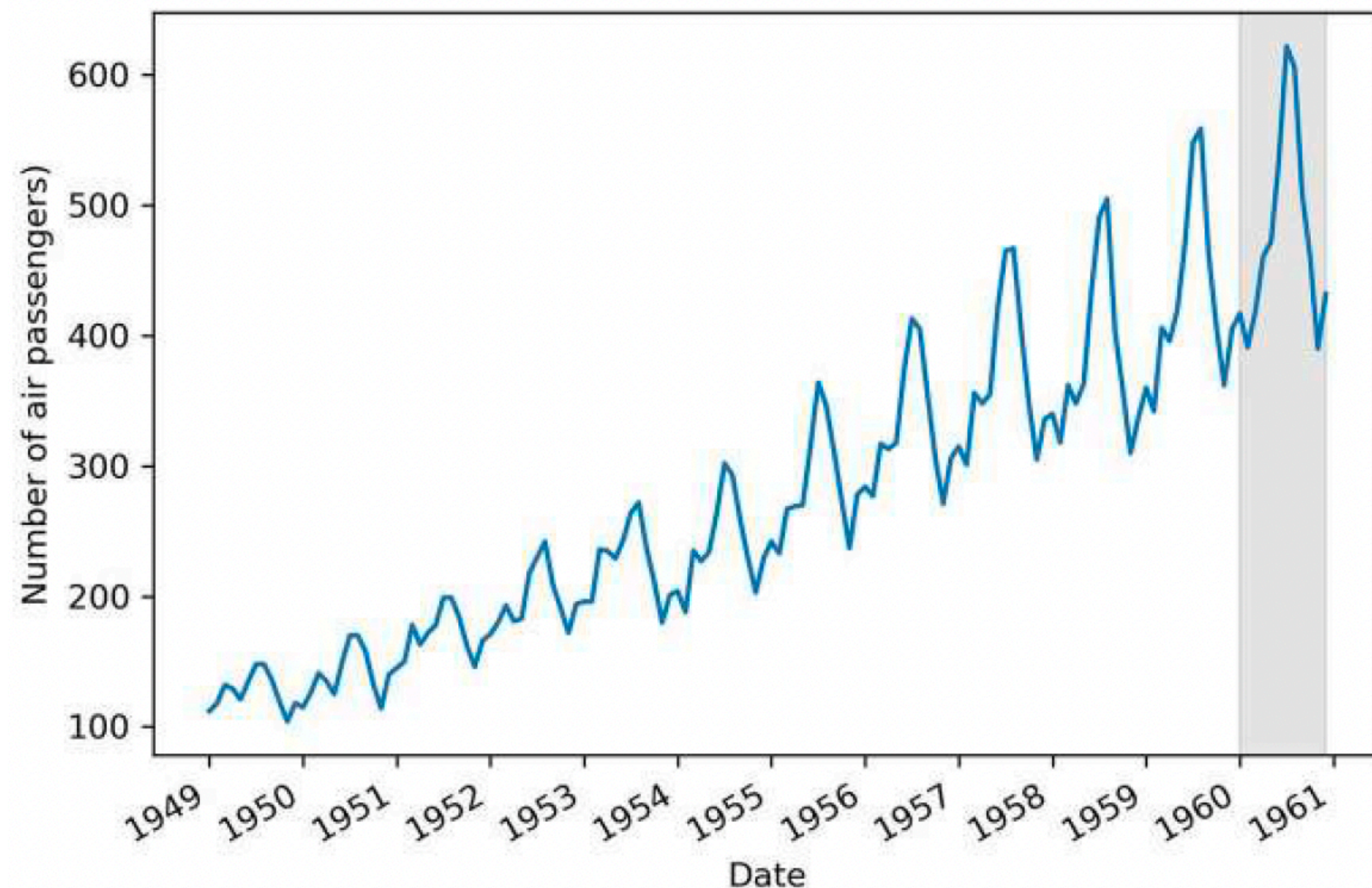
- The **Akaike information criterion (AIC)** is a measure of the quality of a model in relation to other models. It is used for model selection.
- The AIC is a function of the number of parameters k in a model and the maximum value of the likelihood function $\hat{\mathcal{L}}$:

$$AIC = 2k - 2\ln(\hat{\mathcal{L}})$$

- The lower the value of the AIC, the better the model. Selecting according to the AIC allows us to keep a balance between the complexity of a model and its goodness of fit to the data.

Number of monthly air passengers (ARIMA): AIC

- We'll specifically choose a range from 0 to 12 to allow the ARIMA model to go back 12 timesteps in time. Since the data is sampled monthly and we know it is seasonal, we can hypothesise that the number of air passengers in January of a given year is likely predictive of the number of air passengers in January of the following year.



ps = range(0, 13, 1)

qs = range(0, 13, 1)

Ps = [0]

Qs = [0]

d = 2

D = 0

m = 12

ARIMA_order_list = list(product(ps, qs, Ps, Qs))

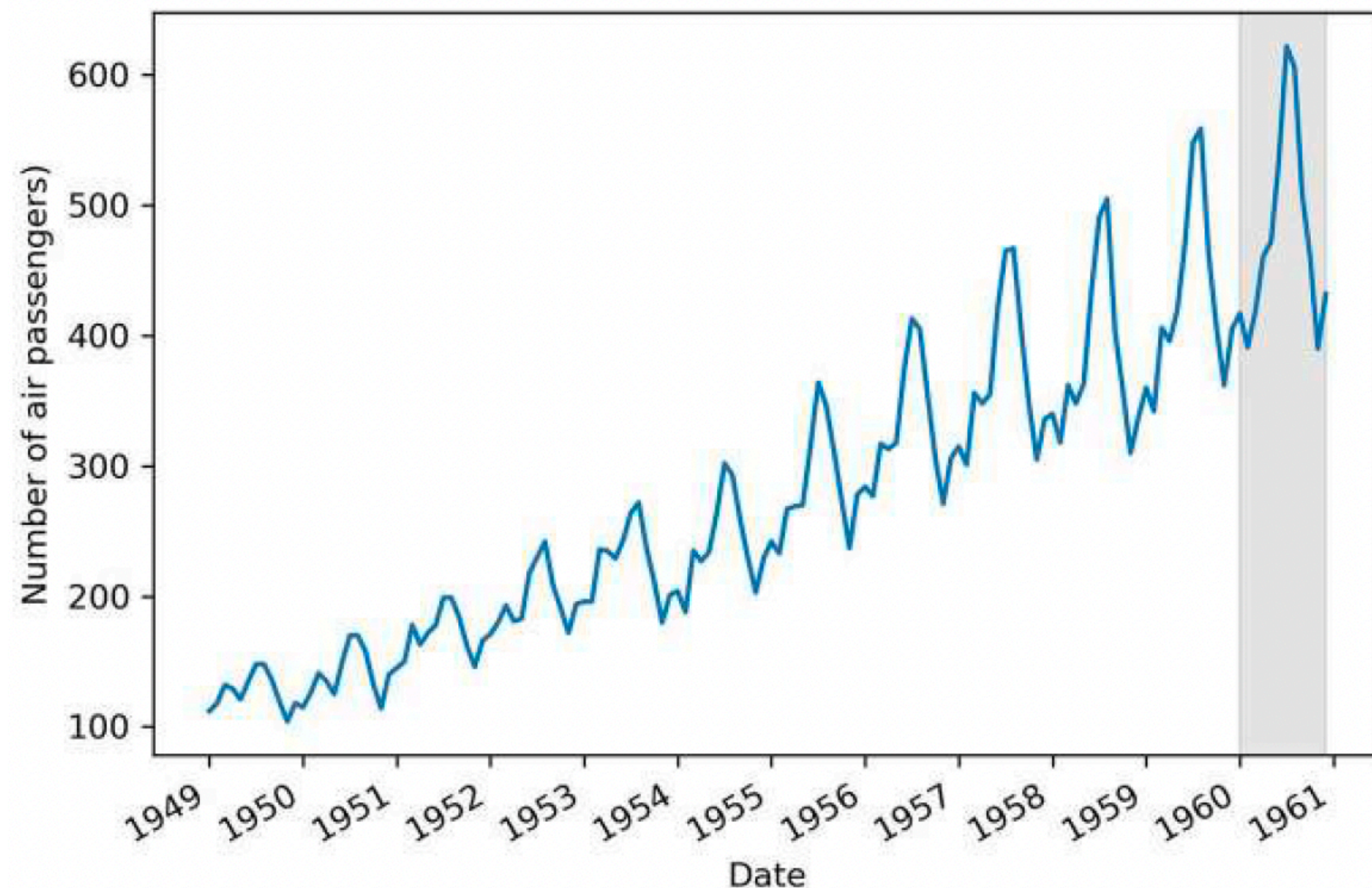
Number of monthly air passengers (ARIMA): AIC

- Fit all unique ARIMA(p, d, q) models:

```
from typing import Union  
from tqdm import tqdm_notebook  
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

```
def optimize_SARIMA(endog: Union[pd.Series, list], order_list: list, d: int, D: int, m: int)  
-> pd.DataFrame:  
    results = []  
    for order in tqdm_notebook(order_list):  
        try:  
            model = SARIMAX(endog, order=(order[0], d, order[1]),  
                           seasonal_order=(order[2], D, order[3], m),  
                           simple_differencing=False).fit(dispatch=False)  
        except:  
            continue  
            aic = model.aic  
            results.append([order, aic])  
result_df = pd.DataFrame(results)  
result_df.columns = ['(p,q,P,Q)', 'AIC']
```

```
#Sort in ascending order, lower AIC is better  
result_df = result_df.sort_values(by='AIC',  
                                ascending=True).reset_index(drop=True)  
return result_df
```



Number of monthly air passengers (ARIMA): AIC

	(p,q,P,Q)	AIC
0	(11, 3, 0, 0)	1016.882539
1	(11, 4, 0, 0)	1019.013048
2	(11, 5, 0, 0)	1020.428000
3	(12, 0, 0, 0)	1020.528594
4	(11, 1, 0, 0)	1021.028226
...	...	...
164	(5, 0, 0, 0)	1281.732157
165	(3, 0, 0, 0)	1300.282335
166	(2, 0, 0, 0)	1302.913196
167	(1, 0, 0, 0)	1308.152194
168	(0, 0, 0, 0)	1311.919269

- Fit all unique $ARIMA(p, d, q)$ models:

train = df['Passengers'][:-12]

ARIMA_result_df = optimize_SARIMA(train, ARIMA_order_list, d, D, m)

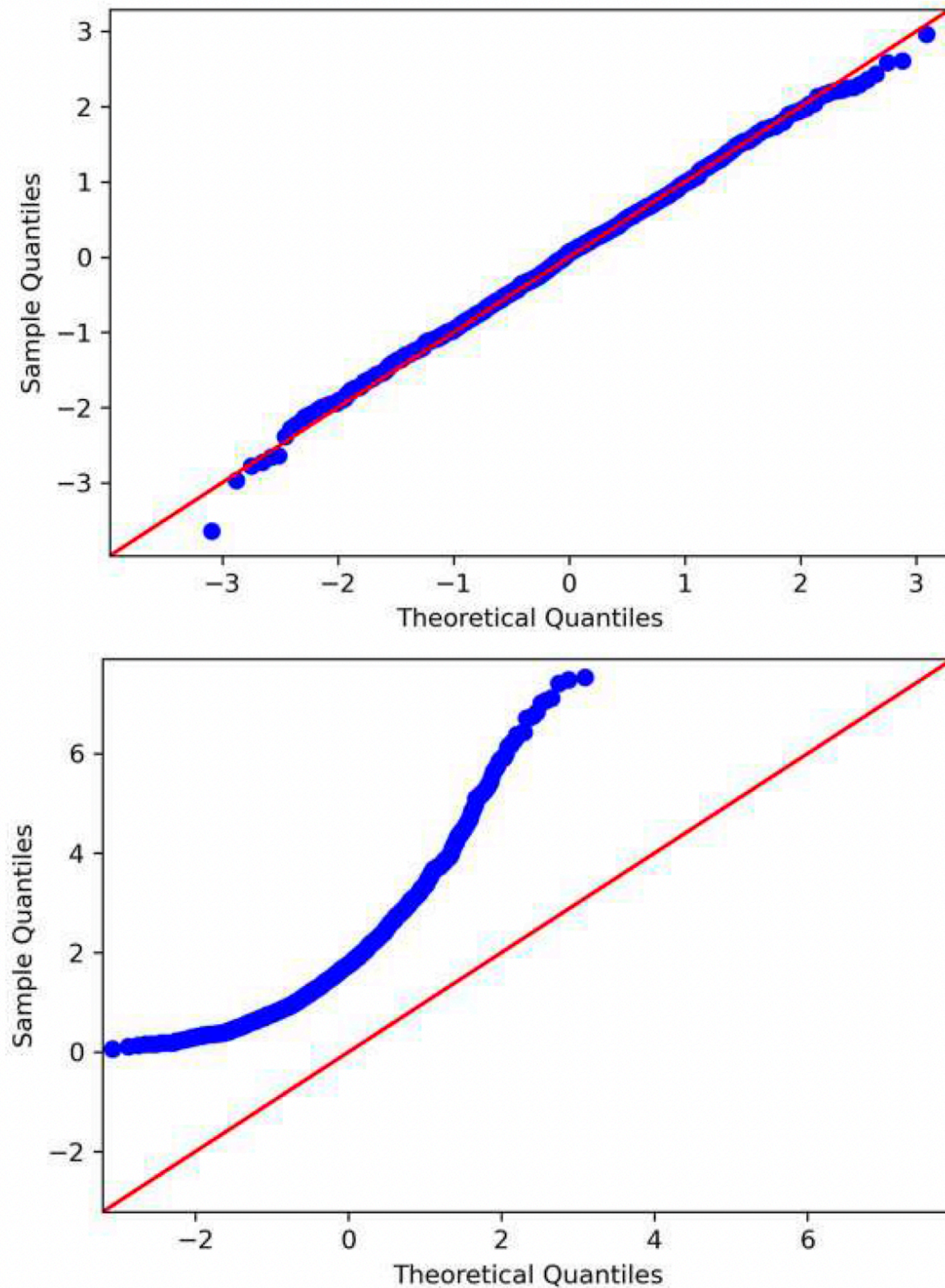
ARIMA_result_df

Number of monthly air passengers (ARIMA): AIC

	(p,q,P,Q)	AIC
0	(11, 3, 0, 0)	1016.882539
1	(11, 4, 0, 0)	1019.013048
2	(11, 5, 0, 0)	1020.428000
3	(12, 0, 0, 0)	1020.528594
4	(11, 1, 0, 0)	1021.028226
...	...	...
164	(5, 0, 0, 0)	1281.732157
165	(3, 0, 0, 0)	1300.282335
166	(2, 0, 0, 0)	1302.913196
167	(1, 0, 0, 0)	1308.152194
168	(0, 0, 0, 0)	1311.919269

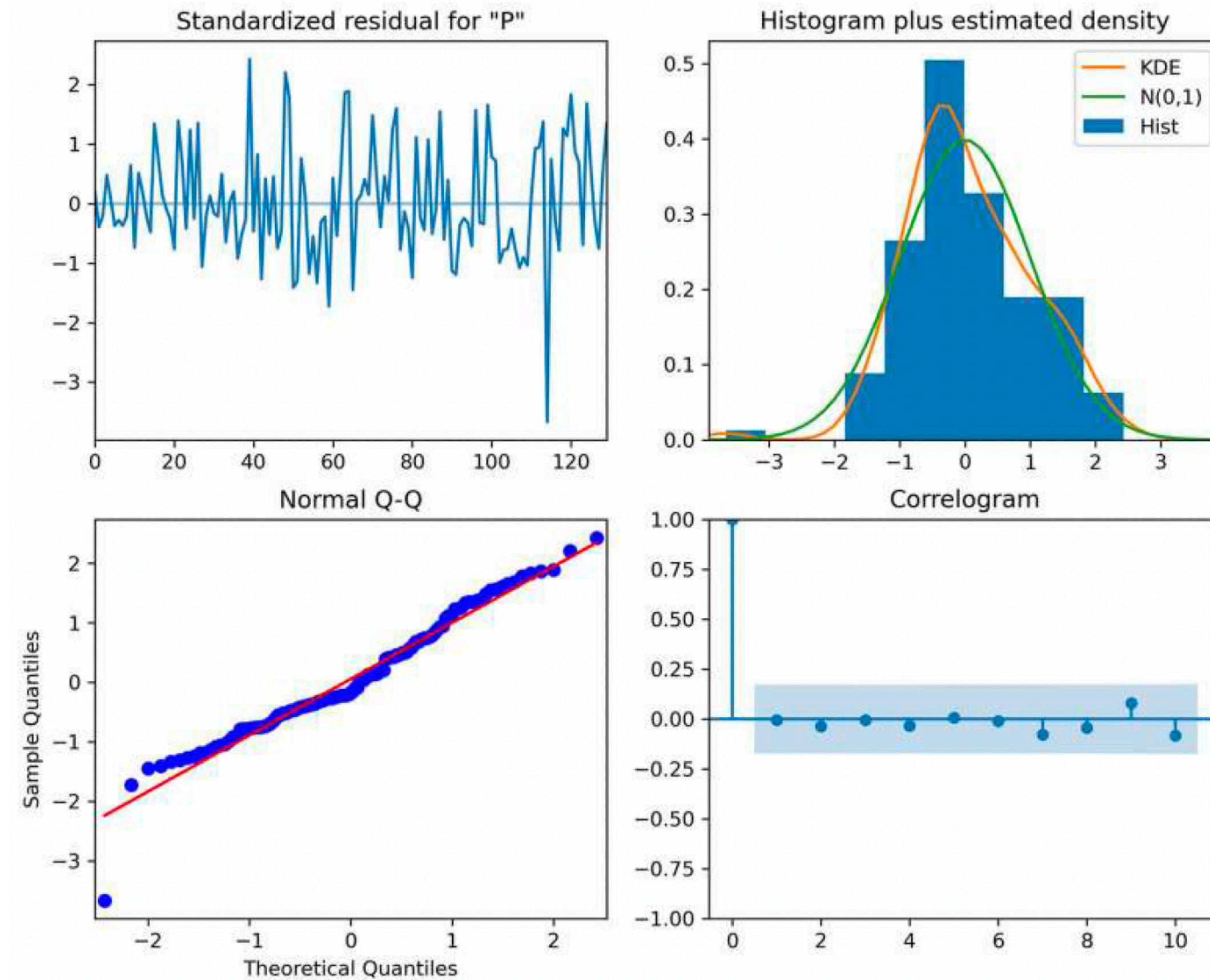
- The model with the lowest AIC is a SARIMA(11,2,3)(0,0,0)₁₂ model, which is equivalent to an **ARIMA(11,2,3)** model.

Residual Analysis: Q-Q plot



- The **quantile-quantile plot (Q-Q plot)** is a graphical tool for verifying our hypothesis that the model's residuals are normally distributed.
- The Q-Q plot is constructed by plotting the quantiles of our residuals on the y -axis against the quantiles of a theoretical distribution, in this case the normal distribution, on the x -axis.
- If both distributions are similar, meaning that the distribution of the residuals is close to a normal distribution, the Q-Q plot will display a straight line that approximately lies on $y = x$. This in turn means that our model is a good fit for our data.

Number of monthly air passengers (ARIMA): Q-Q plot

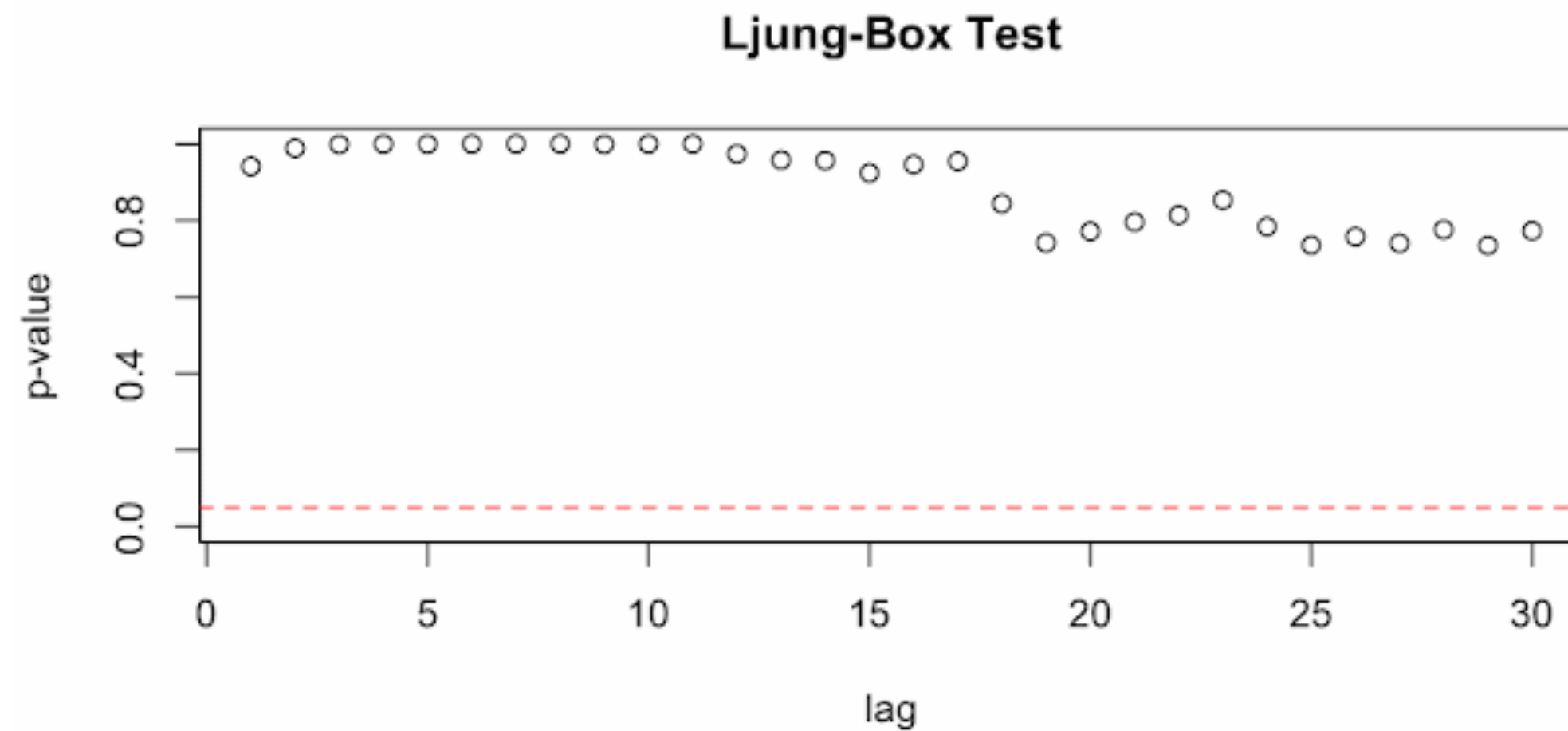


- Use the **plot_diagnostics** method to study the Q-Q plot, as well as the other accompanying plots:

***ARIMA_model = SARIMAX(train, order=(11,2,3),
simple_differencing=False)***

***ARIMA_model_fit = ARIMA_model.fit(dispatch=False)
ARIMA_model_fit.plot_diagnostics(figsize=(10,8))***

Residual Analysis: Ljung-Box Test



- The **Ljung-Box test** is a statistical test that determines whether the autocorrelation of a group of data is significantly different from 0.
- We apply the Ljung-Box test on the model's residuals to test whether they are similar to white noise.
- The *null hypothesis* states that there is no autocorrelation.
- If the **p-value is larger than 0.05**, then the residuals are independently distributed.

Number of monthly air passengers (ARIMA): Ljung-Box Test

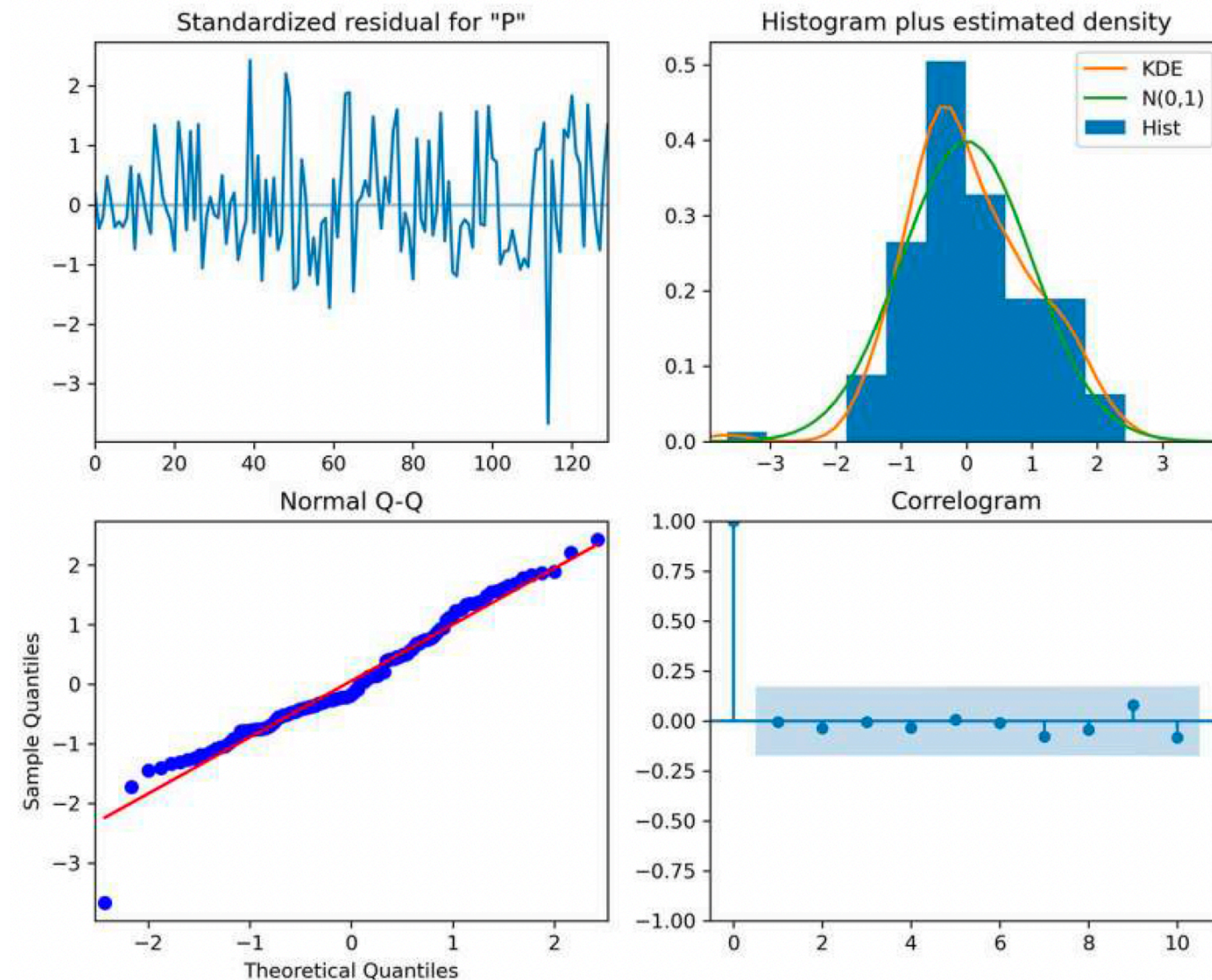
- Our last step is to run the Ljung-Box test on the residuals for the first 10 lags. If the returned p-values exceed 0.05, we cannot reject the null hypothesis, which means that our residuals are uncorrelated and independently distributed, just like white noise.

residuals = ARIMA_model_fit.resid

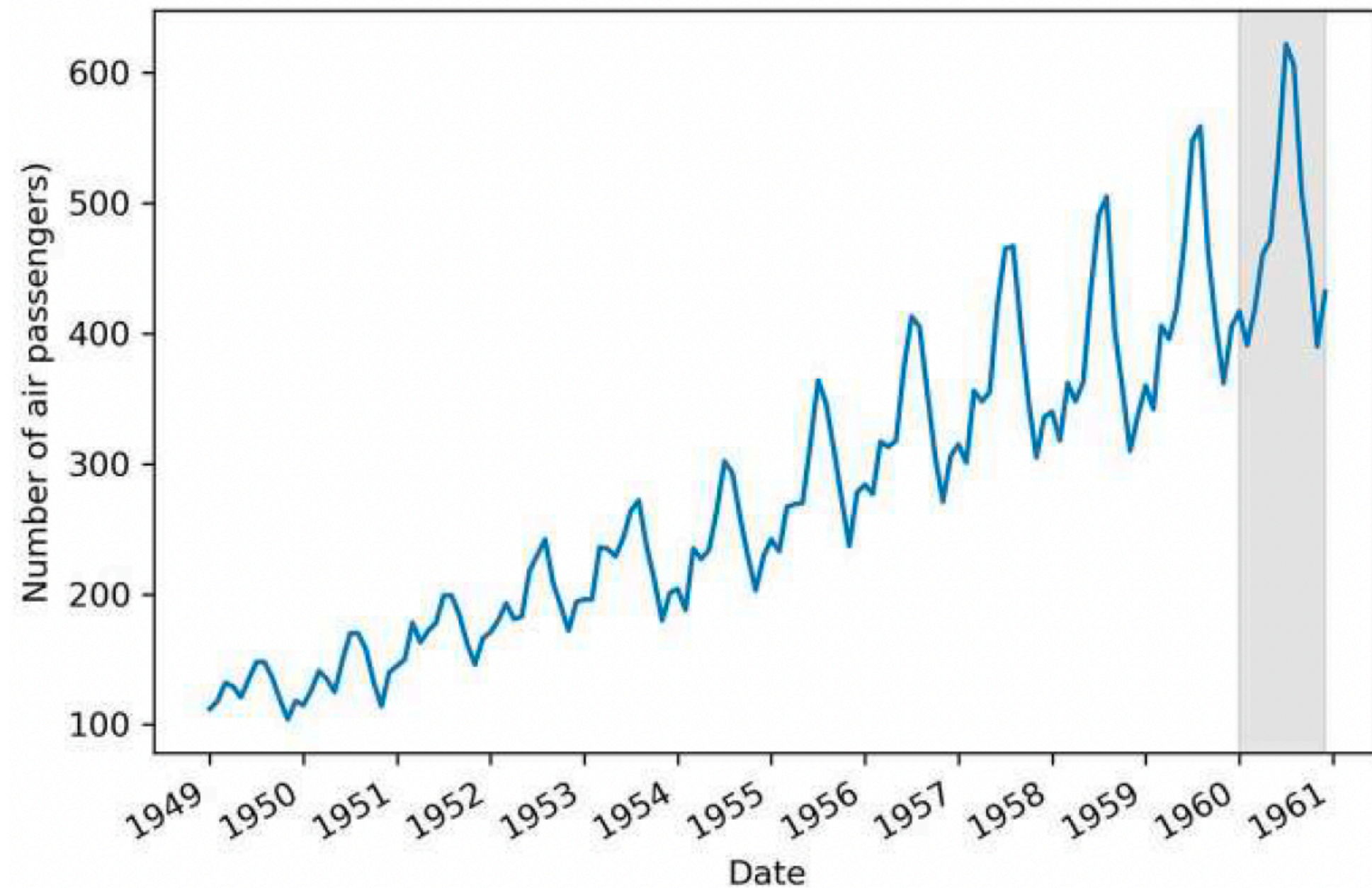
lb = acorr_ljungbox(residuals, np.arange(1, 11, 1))

print(lb)

- The returned p-values are all greater than 0.05 except for the first two values. Interesting: residuals leads us to conclude that they resemble white noise, but the Ljung-Box test points to some correlation at lags 1 and 2. **This means that our ARIMA model is not capturing all the information from the data.**



Number of monthly air passengers (ARIMA): Predictions



- We can append the forecasts from our **ARIMA(11,2,3)** model to the test DataFrame:

```
test = df.iloc[-12:]
```

```
ARIMA_pred = ARIMA_model_fit.get_prediction(132,  
143).predicted_mean
```

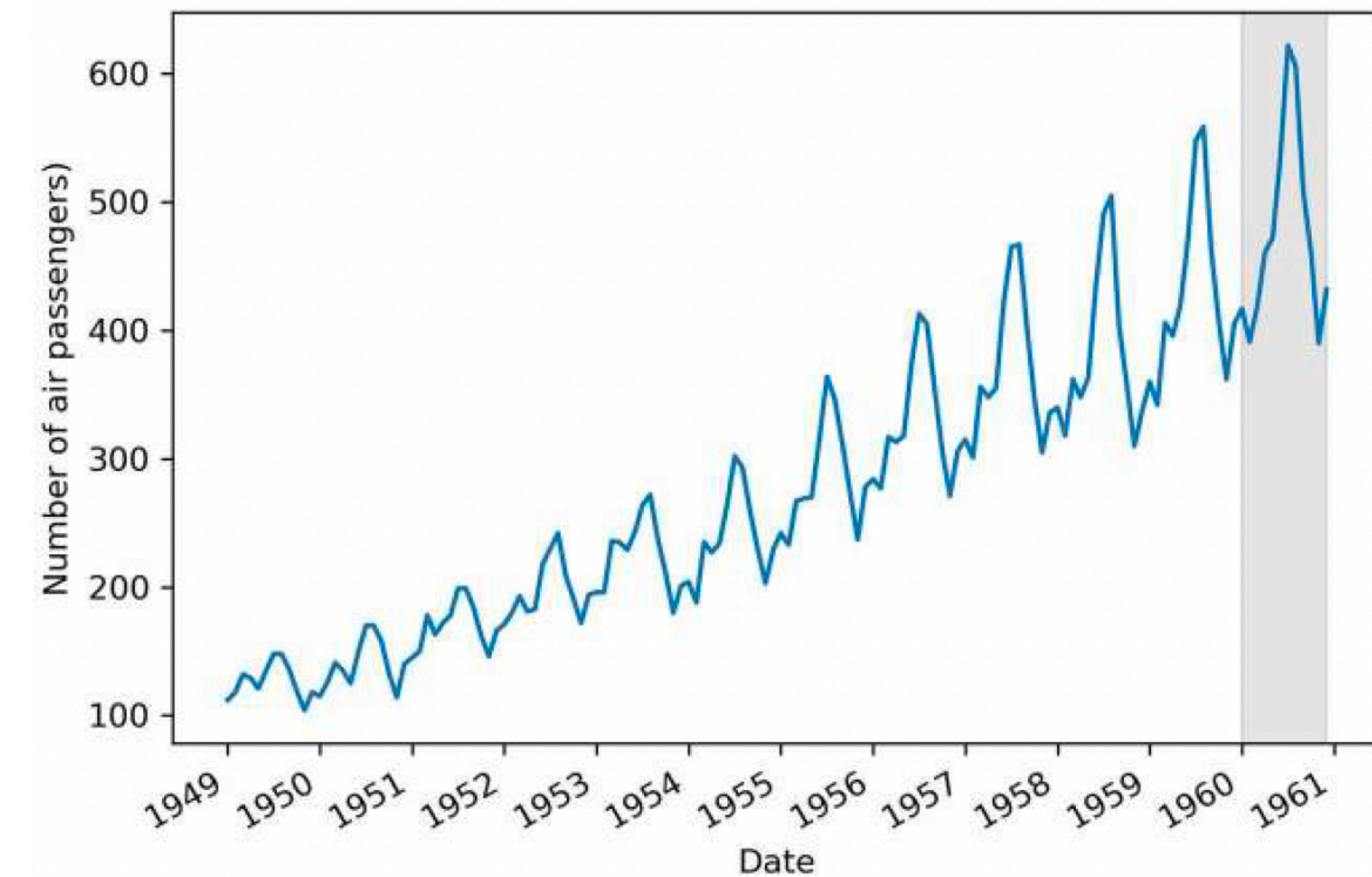
```
test['ARIMA_pred'] = ARIMA_pred
```

- The baseline model is the **naive seasonal forecast**, where we simply use the number of air passengers for each month of 1959 as a forecast for each month of 1960.

```
test['naive_seasonal'] = df['Passengers'].iloc[120:132].values
```


SARIMA

Number of monthly air passengers (SARIMA): Stationarity

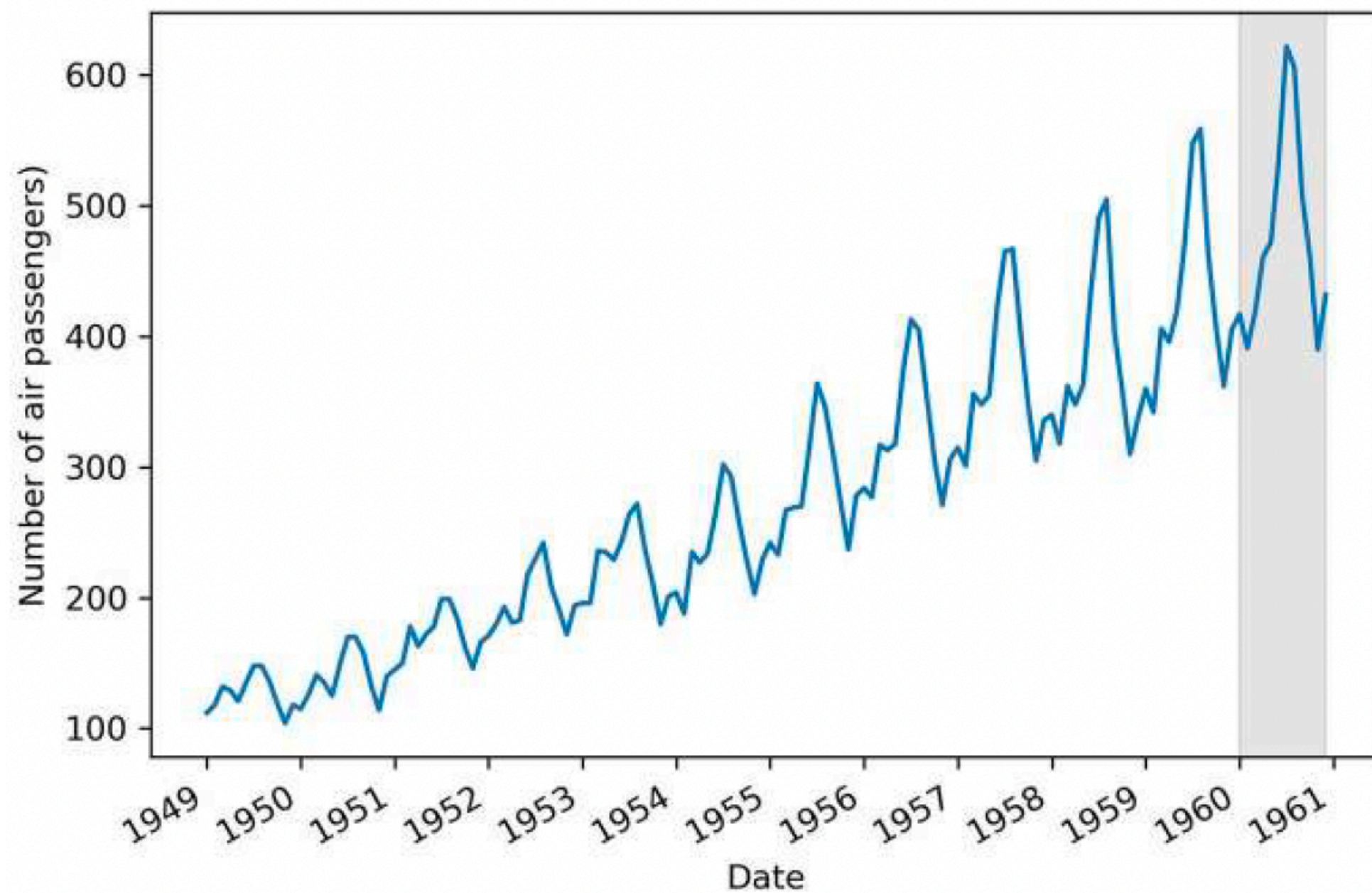


- Use the **adfuller** function from **statsmodels**:

```
from statsmodels.tsa.stattools import adfuller  
ADF_result = adfuller(df['Passengers'])  
print(f'ADF Statistic: {ADF_result[0]}')  
print(f'p-value: {ADF_result[1]}')
```

This results in an ADF statistic of 0.82 and a p-value of 0.99.
Not stationary.

Number of monthly air passengers (SARIMA): Stationarity

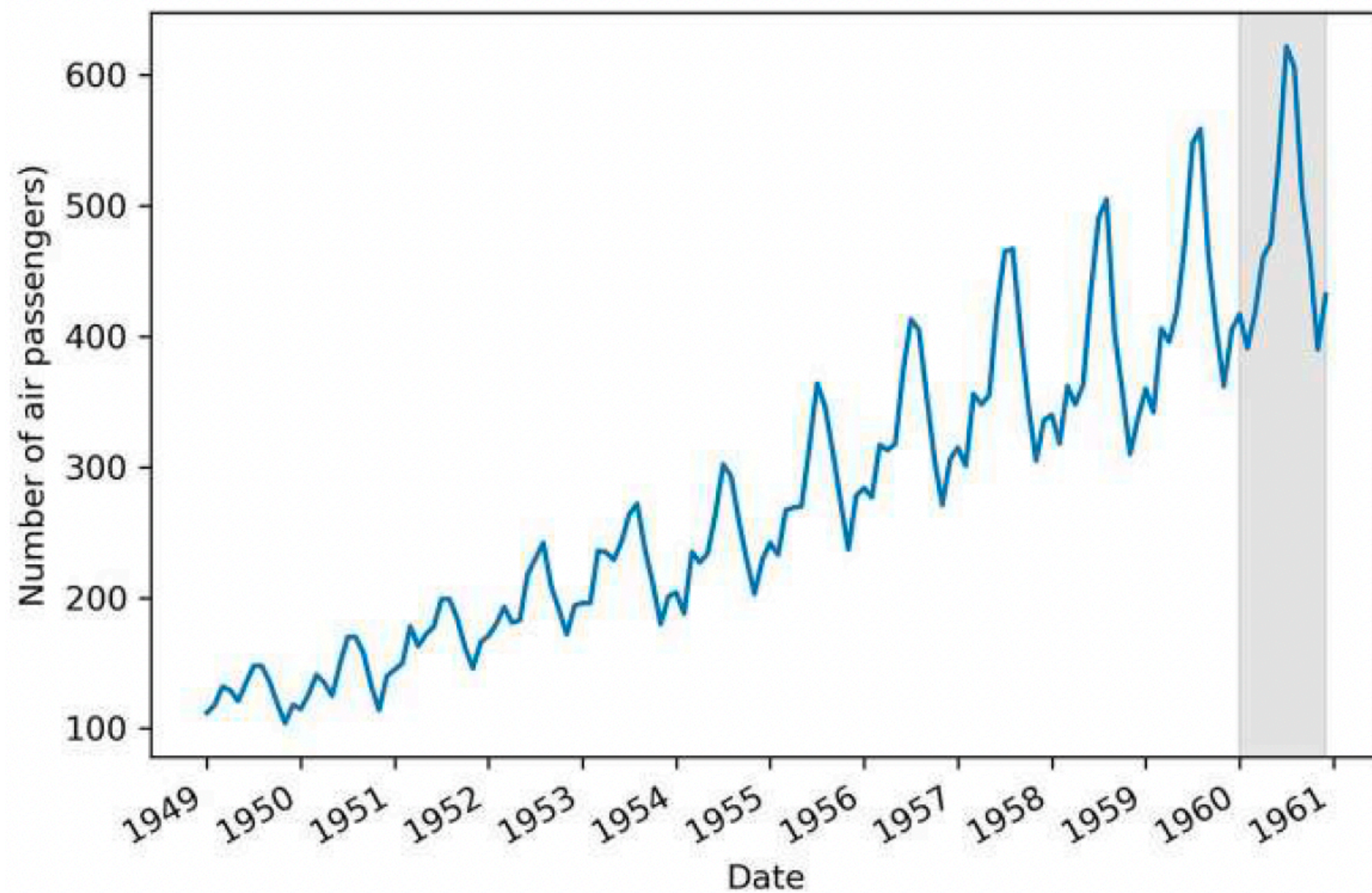


- Apply differencing:

```
df_diff = np.diff(df['Passengers'], n=1)  
ADF_result = adfuller(df_diff)  
print(f'ADF Statistic: {ADF_result[0]}')  
print(f'p-value: {ADF_result[1]}')
```

This results in an ADF statistic of -2.83 and a p-value of 0.054 .
Not stationary.

Number of monthly air passengers (SARIMA): Stationarity



- ❖ Therefore, let's apply a **seasonal difference** and test for stationarity:

```
df_diff_seasonal = np.diff(df_diff, n=12)  
ADF_result = adfuller(df_diff_seasonal)  
print(f'ADF Statistic: {ADF_result[0]}')  
print(f'p-value: {ADF_result[1]}')
```

This results in an ADF statistic of -17.63 and a p-value of 3.82×10^{-30} . *Stationary.*

- ❖ Hence, $d = 1$ and $D = 1$.

Number of monthly air passengers (SARIMA): AIC

- Make a list of (p, q, P, Q) :

$ps = \text{range}(0, 4, 1)$

$qs = \text{range}(0, 4, 1)$

$Ps = \text{range}(0, 4, 1)$

$Qs = \text{range}(0, 4, 1)$

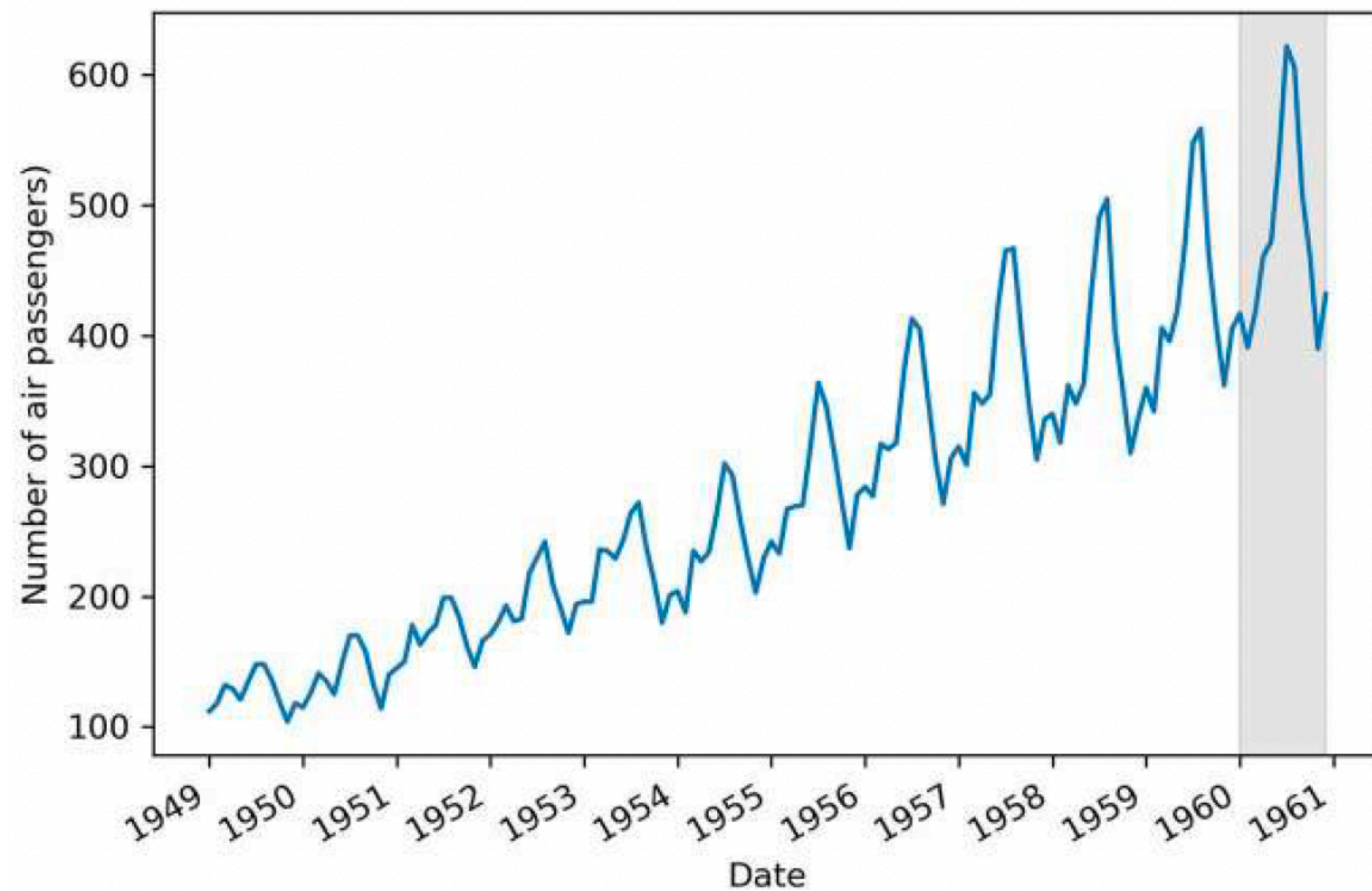
$d = 1$

$D = 1$

$m = 12$

$SARIMA_order_list = \text{list}(\text{product}(ps, qs, Ps, Qs))$

$train = df['Passengers'][:-12]$



Number of monthly air passengers (SARIMA): AIC

	(p,q,P,Q)	AIC
0	(2, 1, 1, 2)	892.244468
1	(2, 1, 2, 1)	893.456064
2	(2, 1, 1, 3)	894.099406
3	(1, 0, 1, 2)	894.286903
4	(0, 1, 1, 2)	894.992179
...	...	...
250	(0, 0, 2, 0)	906.940147
251	(3, 2, 0, 3)	907.181875
252	(0, 0, 3, 2)	907.464271
253	(0, 0, 3, 0)	908.742583
254	(0, 0, 0, 3)	908.781405

- Fit all unique $SARIMA(p, d, q)(P, D, Q)_m$ models:

train = df['Passengers'][:-12]

SARIMA_result_df = optimize_SARIMA(train, SARIMA_order_list, d, D, m)

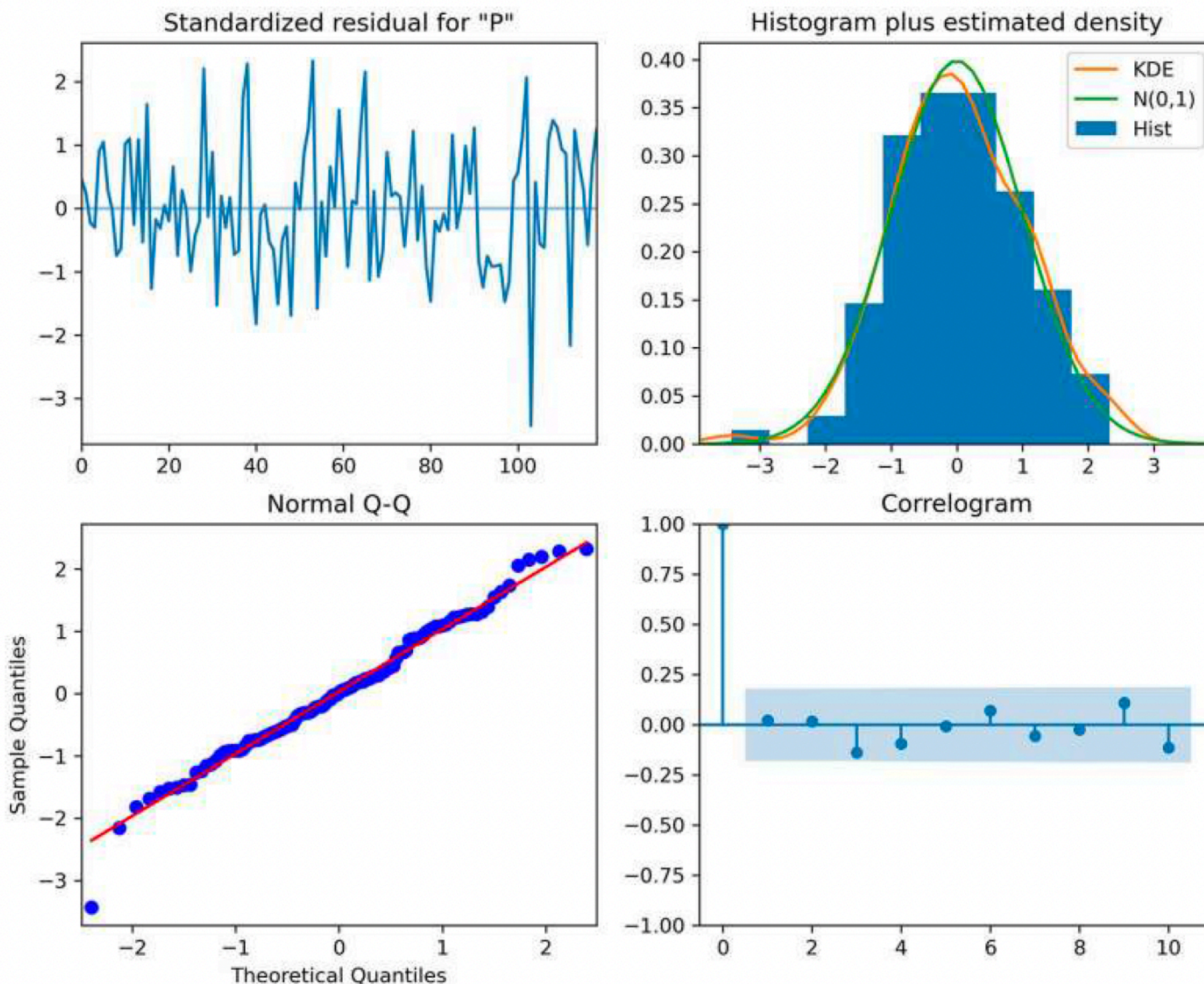
SARIMA_result_df

Number of monthly air passengers (SARIMA): AIC

	(p,q,P,Q)	AIC
0	(2, 1, 1, 2)	892.244468
1	(2, 1, 2, 1)	893.456064
2	(2, 1, 1, 3)	894.099406
3	(1, 0, 1, 2)	894.286903
4	(0, 1, 1, 2)	894.992179
...	...	...
250	(0, 0, 2, 0)	906.940147
251	(3, 2, 0, 3)	907.181875
252	(0, 0, 3, 2)	907.464271
253	(0, 0, 3, 0)	908.742583
254	(0, 0, 0, 3)	908.781405

- The model with the lowest AIC is a **SARIMA(2,1,1)(1,1,2)₁₂** model.

Number of monthly air passengers (SARIMA): Q-Q plot



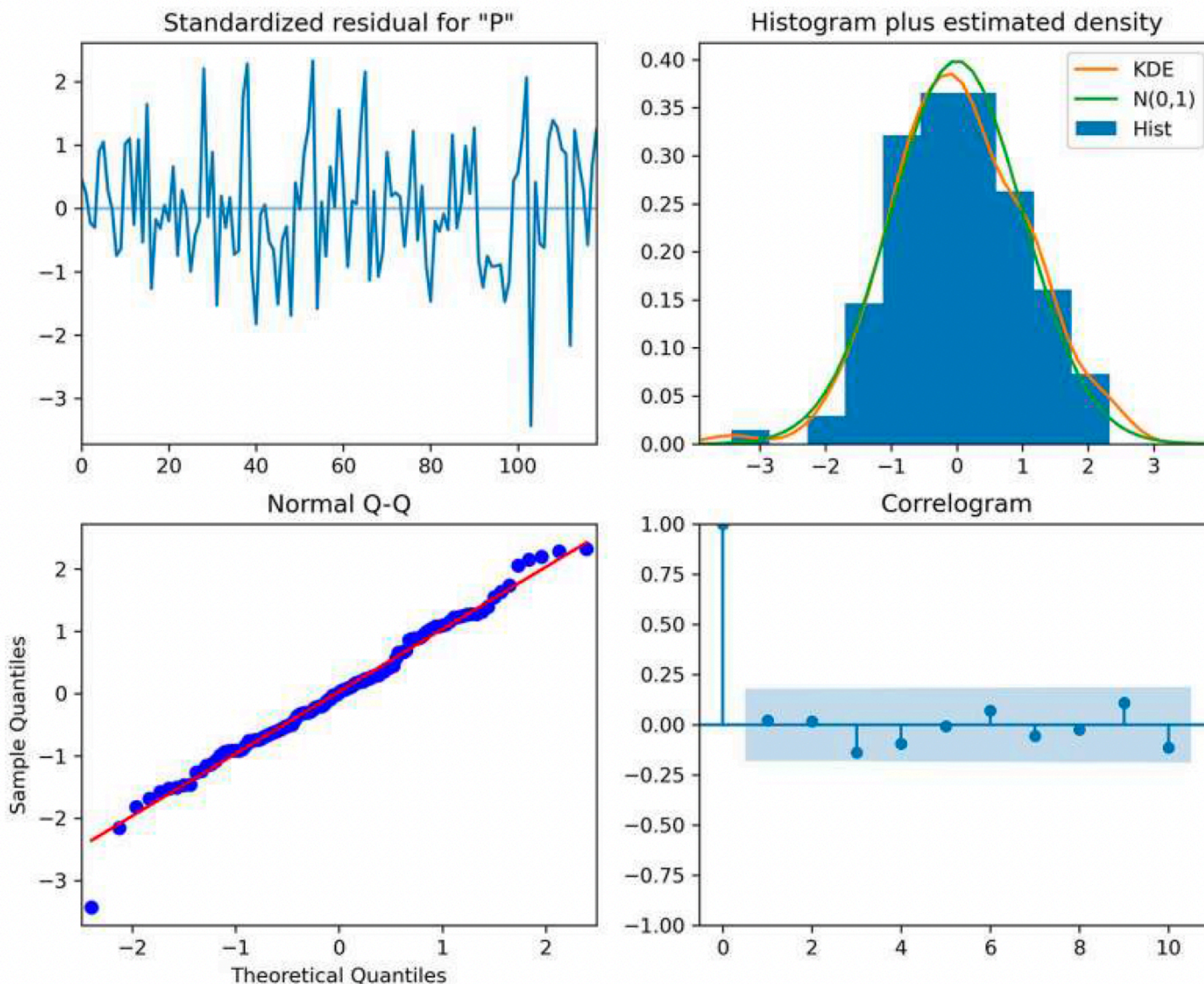
- Use the **plot_diagnostics** method to study the Q-Q plot, as well as the other accompanying plots:

***SARIMA_model = SARIMAX(train, order=(2,1,1),
seasonal_order=(1,1,2,12), simple_differencing=False)***

SARIMA_model_fit = SARIMA_model.fit(dispatch=False)

SARIMA_model_fit.plot_diagnostics(figsize=(10,8))

Number of monthly air passengers (SARIMA): Ljung-Box Test



- Our last step is to run the Ljung-Box test on the residuals for the first 10 lags. If the returned p-values exceed 0.05, we cannot reject the null hypothesis, which means that our residuals are uncorrelated and independently distributed, just like white noise.

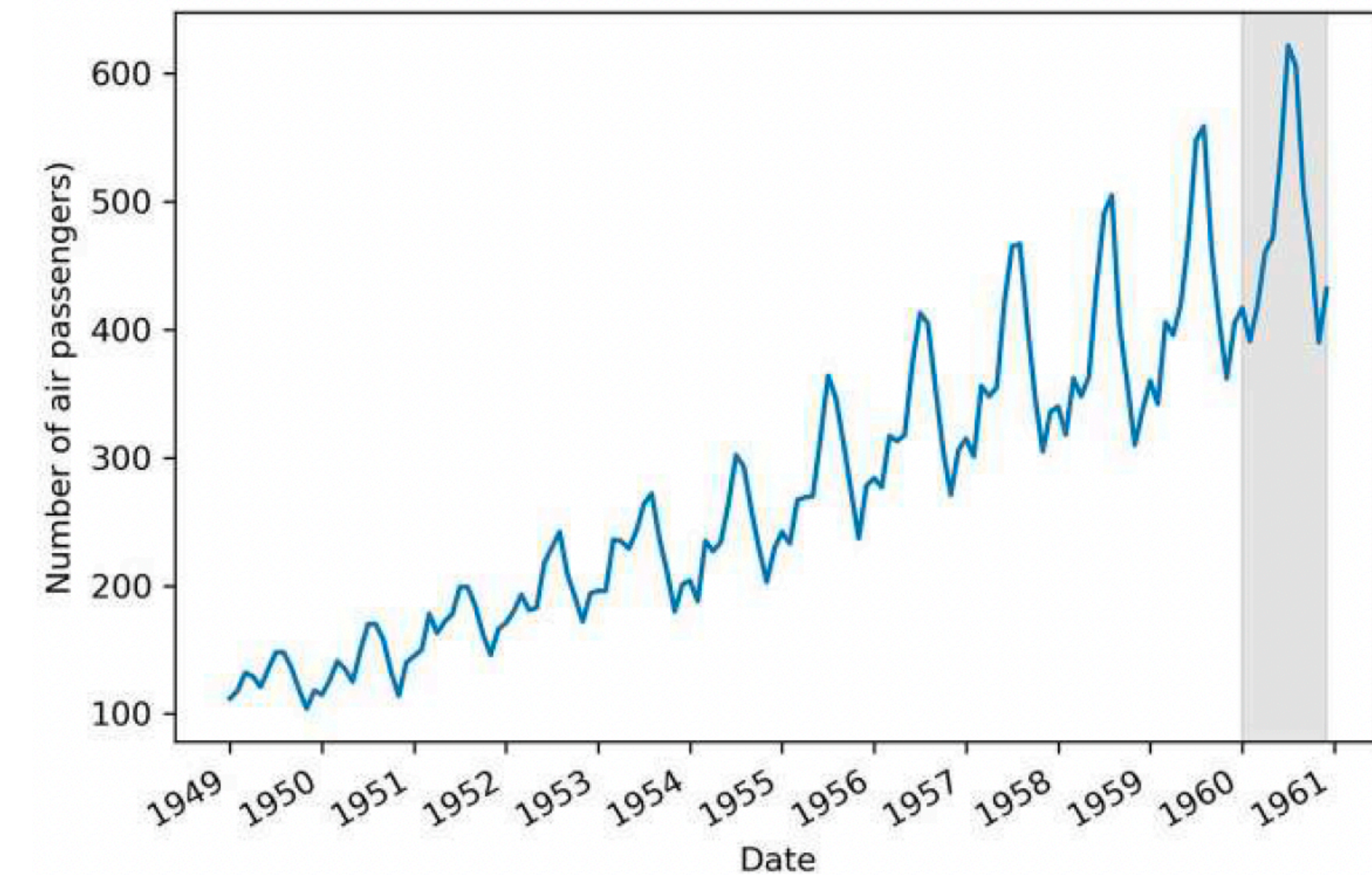
residuals = SARIMA_model_fit.resid

lb = acorr_ljungbox(residuals, np.arange(1, 11, 1))

print(lb)

- The returned p-values are all greater than 0.05. Therefore, we do not reject the null hypothesis, and we conclude that **the residuals are independent and uncorrelated, just like white noise.**

Number of monthly air passengers (SARIMA): Predictions

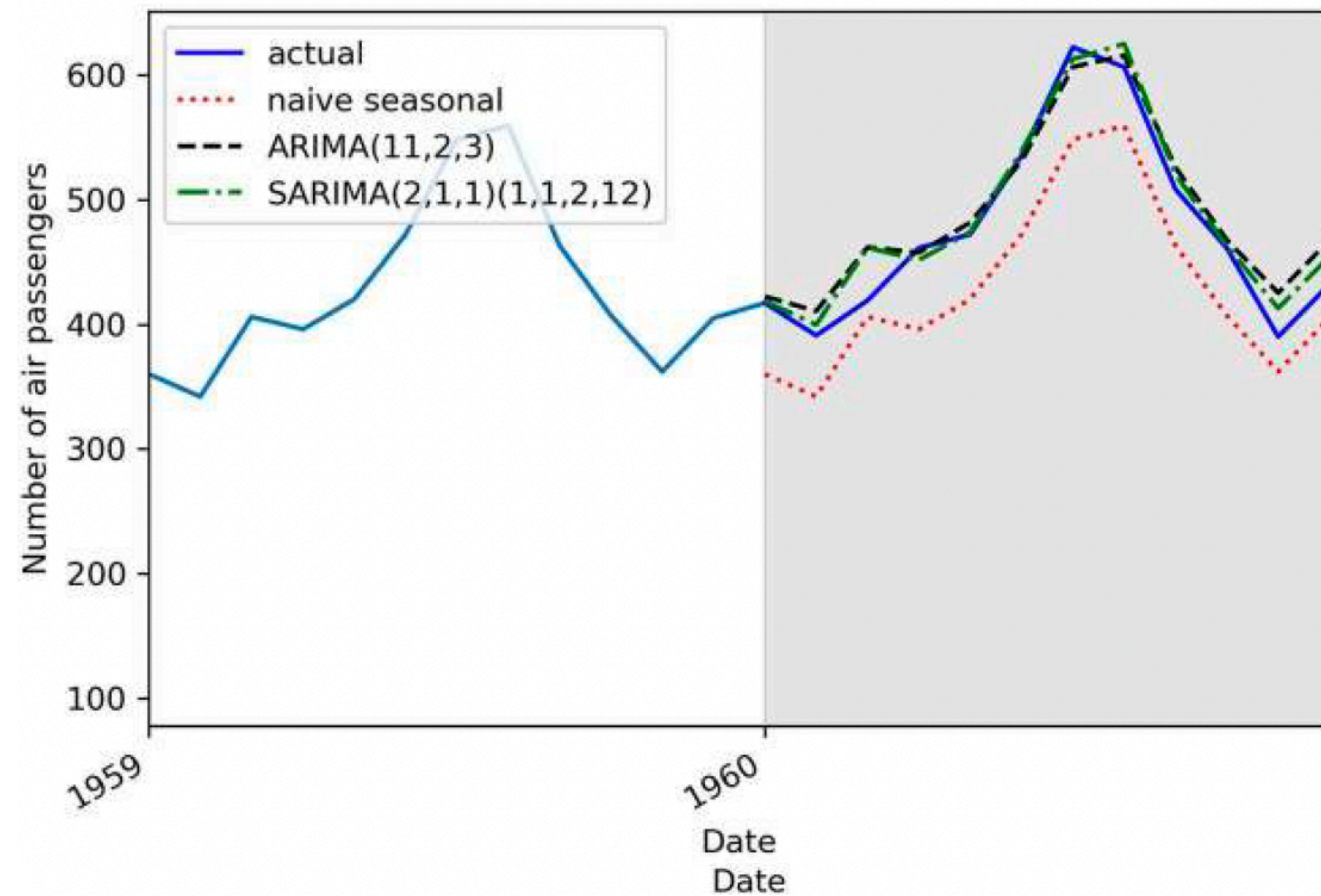


- We can append the forecasts from our **SARIMA(2,1,1)(1,1,2)₁₂** to the test DataFrame:

SARIMA_pred = SARIMA_model_fit.get_prediction(132, 143).predicted_mean

test['SARIMA_pred'] = SARIMA_pred

Number of monthly air passengers: Comparison



- Measure the MAPE:

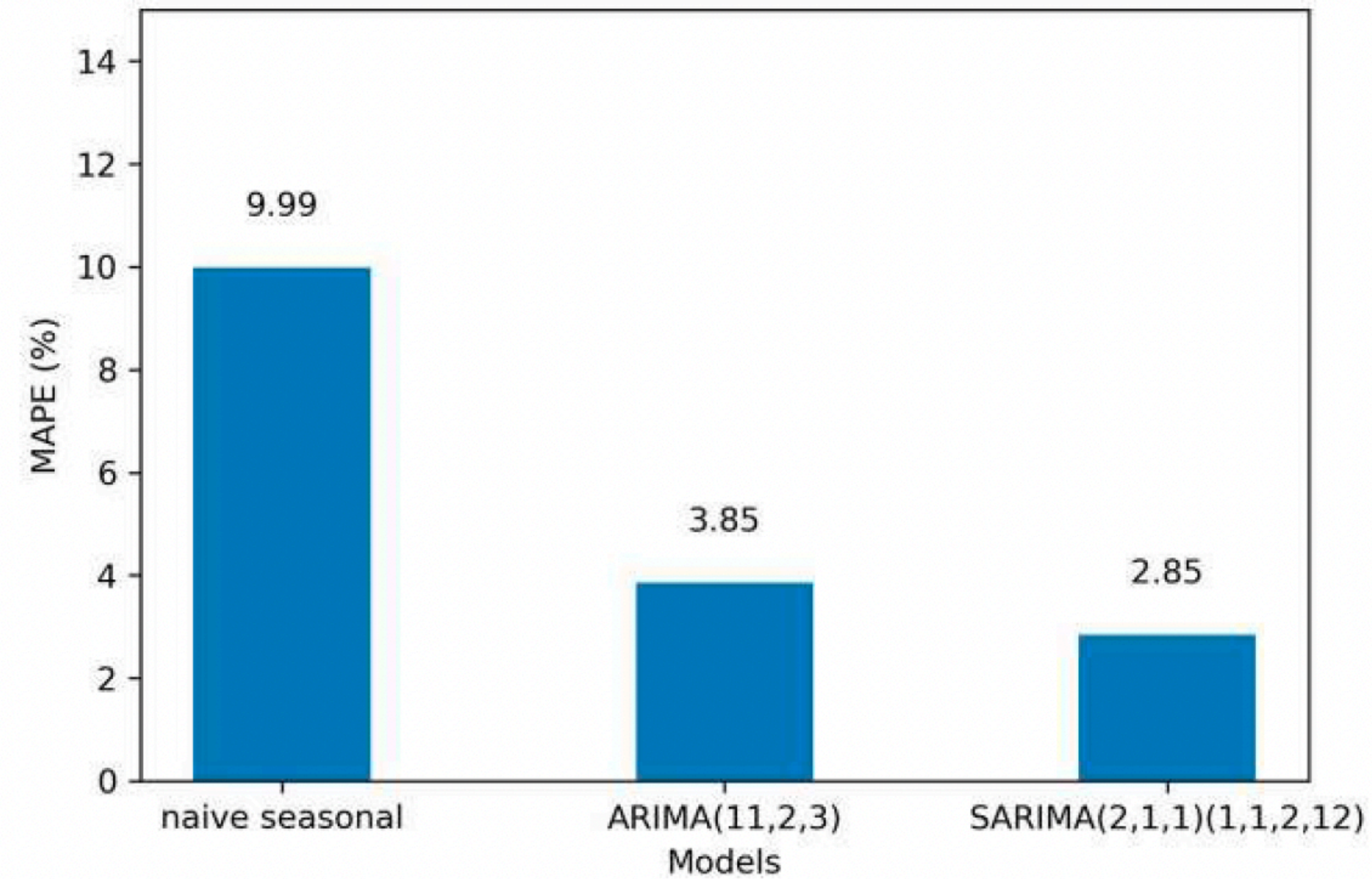
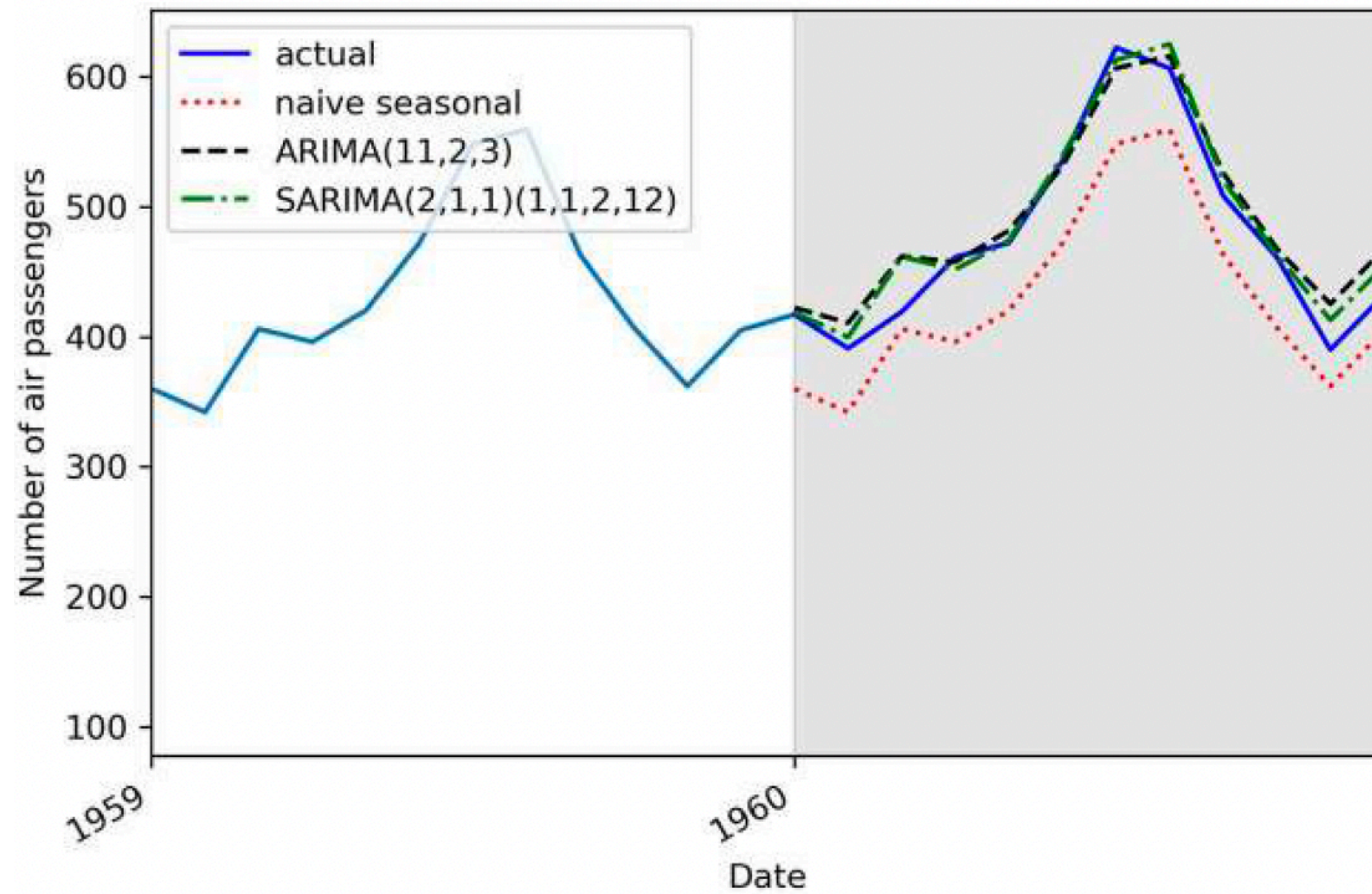
```
def mape(y_true, y_pred):  
    return np.mean(np.abs((y_true - y_pred) / y_true)) * 100
```

```
mape_naive_seasonal = mape(test['Passengers'],  
test['naive_seasonal'])
```

```
mape_ARIMA = mape(test['Passengers'],  
test['ARIMA_pred'])
```

```
mape_SARIMA = mape(test['Passengers'],  
test['SARIMA_pred'])
```


Number of monthly air passengers: Comparison



Thank you!